AsistenteIA

Asistente académico móvil con inteligencia artificial — Documentación del proyecto final · .NET MAUI, API REST y SQLite

Integrantes: Juan Carlos Díaz Cifuentes · Sebastián Grande Moreno

Docente: Sandra Esperanza Sánchez Barragán

Universidad de San Buenaventura · Tecnología en Desarrollo de Software · VI semestre · Programación Avanzada y Aplicada · Noviembre de 2026

Tabla de contenido

1. Introducción 6

1.1 Propósito 6

1.2 Problema 6

1.3 Alcance 6

1.4 Usuario objetivo 6

1.5 Equipo 6

2. Tecnologías utilizadas 7

2.1 Fuentes 8

3. Requisitos técnicos 9

3.1 SDK y plataformas 9

3.2 Herramientas 9

3.3 Paquetes NuGet 10

3.4 Servicios externos 10

4. Requisitos y análisis 11

4.1 Historias de usuario y criterios de aceptación 11

4.2 Requisitos funcionales 12

4.3 Requisitos no funcionales 13

4.4 Funcionalidades implementadas 14

5. Arquitectura del sistema 16

5.1 Diagrama de arquitectura 16

5.2 Componentes 16

5.3 Comunicación entre componentes 17

5.4 Estructura de la solución 17

5.5 Recorrido de una consulta 18

5.6 Inyección de dependencias y navegación 18

5.7 Decisiones técnicas..... 19
6. Diagrama de clases..... 20
6.1 Clases principales..... 20
6.2 Relaciones..... 22
7. Arquitectura de proveedores de IA..... 23
7.1 Diseño..... 23
7.2 Características de cada proveedor..... 23
7.3 Selección y cambio manual..... 24
7.4 Cómo agregar otro proveedor..... 24
8. Interfaz de la aplicación..... 25
8.1 Pantalla de inicio..... 25
8.2 Pantalla de consulta..... 25
8.3 Pantalla de proyectos..... 26
8.4 Pantalla de historial..... 27
8.5 Pantalla de perfil de usuario..... 27
8.6 Pantalla de configuración de la API..... 28
8.7 Conexión de la clave de API..... 29
9. Documentación de la API REST..... 30
9.1 Google Gemini — generateContent..... 30
9.2 Pollinations — chat/completions (compatible con OpenAI)..... 31
9.3 Códigos de estado..... 32
9.4 Ejemplo real de consulta..... 32
10. Gestión de errores y excepciones..... 33
10.1 Errores antes de enviar..... 33
10.2 Errores HTTP..... 34
10.3 Problemas de conexión y tiempo..... 34

11. Persistencia con SQLite..... 36
11.1 Modelo entidad-relación..... 36
11.2 Diccionario de datos..... 37
11.3 Operaciones CRUD..... 39
12. Gestión de API Keys..... 41
12.1 Ciclo de vida..... 41
12.2 Dónde se guarda en cada plataforma..... 41
12.3 Protección de credenciales..... 41
13. Seguridad, privacidad y uso responsable de la IA..... 43
13.1 Matriz de riesgos: privacidad, seguridad y uso responsable..... 43
13.2 Auditoría de seguridad de API Keys..... 45
13.3 Declaración de uso de herramientas de IA..... 47
14. Pruebas y calidad..... 48
14.1 Plan de pruebas manuales..... 48
14.2 Evidencias de las pruebas manuales..... 51
14.3 Pruebas automáticas..... 55
15. Manual de usuario..... 57
15.1 Primer uso..... 57
15.2 Pantallas (ver capturas en el capítulo 8)..... 57
15.3 Hacer una consulta..... 58
15.4 Cambiar de proveedor..... 58
15.5 Privacidad y datos..... 58
15.6 Recomendaciones..... 59
16. Manual técnico para desarrolladores..... 60
16.1 Organización del código..... 60
16.2 Patrones y reglas..... 60

16.3 Inyección de dependencias (MauiProgram.cs)....60
16.4 Configuración y ejecución....61
16.5 Pruebas....61
16.6 Mantenimiento....61
17. Manual de solución de problemas....63
17.1 Instalación....63
17.2 Compilación....63
17.3 Conexión con la IA....63
17.4 Configuración y datos....64
18. Historial de cambios del proyecto....65
Responsables por área....66
19. Trabajo en equipo y repositorio Git....67
19.1 Roles....67
19.2 Repositorio....67
19.3 Historial de commits....67
19.4 Flujo de trabajo....68
20. Lista de cotejo de entrega....69
21. Conclusiones y trabajo futuro....70
21.1 Resultados obtenidos....70
21.2 Cumplimiento de objetivos....70
21.3 Limitaciones....70
21.4 Trabajo futuro....71

1. Introducción

1.1 Propósito

AsistenteIA es un asistente académico multiplataforma (Windows y Android) construido con .NET MAUI. Envía consultas académicas a un modelo de inteligencia artificial mediante una API REST, muestra la respuesta, la guarda en un historial local (SQLite) y permite organizarla en proyectos, con controles de seguridad, privacidad y uso responsable.

1.2 Problema

Los estudiantes usan asistentes de IA de forma dispersa: sin organización por materia, sin registro de lo consultado, sin verificar las respuestas y, a veces, compartiendo datos personales con el proveedor.

1.3 Alcance

1. Enviar consultas académicas a un modelo de IA por REST, eligiendo entre **Google Gemini** y **Pollinations**.
2. Guardar las respuestas localmente y agruparlas en proyectos.
3. Promover la verificación de las respuestas (marca «verificada» y notas).
4. Proteger la clave de API y la privacidad del estudiante.

Fuera de alcance: cuentas de usuario en un servidor, sincronización en la nube, conversaciones de varios turnos, carga de archivos y generación de imágenes.

1.4 Usuario objetivo

Estudiante de educación superior que necesita aclarar, resumir o practicar contenidos con ayuda de IA de forma rápida y segura.

1.5 Equipo

Integrante	Rol
Juan Carlos Díaz Cifuentes	Servicios y datos; calidad y documentación
Sebastián Grande Moreno	Líder e integración; interfaz y accesibilidad

Docente: Sandra Esperanza Sánchez Barragán · **Asignatura:** Programación Avanzada y Aplicada · VI semestre · Tecnología en Desarrollo de Software · Universidad de San Buenaventura.

2. Tecnologías utilizadas

Versiones tomadas del código del proyecto y verificadas en las fuentes oficiales el 3 de noviembre de 2026.

Componente	Versión / detalle	Uso
.NET SDK	**9.0.100** (global.json, latestFeature) · ⚠️ fin de soporte 10/11/2026 → migrar a .NET 10 LTS	Compilación.
Visual Studio y .NET MAUI	Workload maui de .NET 9 ($(MauiVersion))	Interfaz multiplataforma.
Plataformas	net9.0-windows10.0.19041.0 (mín. 10.0.17763) y net9.0-android (mín. API 21)	Destinos.
C# y XAML	C# 13 (por defecto en .NET 9): *collection expressions* y expresiones switch con patrones	Lógica e interfaz.
MVVM — CommunityToolkit.Mvvm	**8.3.2**	[ObservableProperty], [RelayCommand].
Shell	TabBar de 5 pestañas + 3 rutas secundarias	Navegación.
HTTP/JSON — HttpClient y System.Text.Json	HttpClient singleton (60 s) · System.Net.Http.Json · JsonDocument para errores	Consumo REST.
SQLite — sqlite-net-pcl	**1.9.172** + SQLitePCLRaw.bundle_green **2.1.10**	Historial y proyectos.
Almacenamiento seguro	SecureStorage de MAUI (Keystore / DataProtection) + Preferences	Claves y preferencias.
Registro	Microsoft.Extensions.Logging.Debug **9.0.0** (solo DEBUG)	Depuración.
Pruebas — AsistenteIA.Tests	**xUnit 2.9.2**, xunit.runner.visualstudio 2.8.2, Microsoft.NET.Test.Sdk 17.11.1	66 pruebas.
IA – Google Gemini	API v1beta generateContent. Modelos estables: gemini-3.5-flash-lite, gemini-3.1-flash-lite, gemini-3.6-flash, gemini-3.8-flash (todos vigentes; gemini-2.0-* y gemini-3.1-flash-lite-preview ya fueron retirados)	Proveedor 1.
IA – Pollinations	gen.pollinations.ai/v1/chat/completions (formato OpenAI), clave en Authorization: Bearer, modelo por defecto openai	Proveedor 2.
Control de	Git + GitHub sebitazz321/ASISTENTEIA	Repositorio.

Componente	Versión / detalle	Uso
versiones		
Asistencia de IA para desarrollo	Claude (Anthropic) (ver declaración de uso de IA, capítulo 13)	Apoyo en código y documentación.

2.1 Fuentes

- Modelos Gemini: https://ai.google.dev/gemini-api/docs/models
- Pollinations API: https://gen.pollinations.ai/docs y https://github.com/pollinations/pollinations/blob/master/APIDOCS.md
- Fin de soporte de .NET 8 y 9: https://devblogs.microsoft.com/dotnet/dotnet-8-9-end-of-support/
- Política de soporte de .NET MAUI: https://dotnet.microsoft.com/platform/support/policy/maui

3. Requisitos técnicos

Versiones tomadas de AsistenteIA.csproj, AsistenteIA.Tests.csproj y global.json.

3.1 SDK y plataformas

Elemento	Valor en el proyecto	Notas
.NET SDK	**9.0.100** (global.json, rollForward: latestFeature)	Acepta cualquier SDK 9.0.1xx o superior de la banda 9.0.
Frameworks de destino	net9.0-android · net9.0-windows10.0.19041.0	iOS y Mac Catalyst tienen carpeta en Platforms, pero no se compilan.
.NET MAUI	$(MauiVersion) (lo fija la carga de trabajo maui del SDK 9)	—
Windows (mínimo)	10.0.17763.0 (Windows 10 versión 1809)	WindowsPackageType = None (app no empaquetada).
Android (mínimo)	API 21 (Android 5.0)	Compilado con el SDK de Android del workload.
Pruebas	net9.0 (sin MAUI)	Se ejecutan en cualquier equipo con el SDK.

> ⚠️ **Fin de soporte:** Microsoft finaliza el soporte de **.NET 8 y .NET 9 el 10 de noviembre de 2026**. Se recomienda migrar a **.NET 10 (LTS, soporte hasta noviembre de 2028)**: cambiar los TFM a net10.0-... y global.json a 10.0.100. Ver el capítulo 21.

3.2 Herramientas

Herramienta	Versión mínima	Carga de trabajo / componente
Windows	10 (1809) u 11	—
Visual Studio 2022	17.12 o superior (compatible con .NET 9)	«Desarrollo de la interfaz de usuario de aplicaciones multiplataforma de .NET» (.NET MAUI)
Componentes adicionales	—	Herramientas de C++ para escritorio (requeridas por SQLitePCLRaw en Windows)
CLI (opcional)	dotnet workload install maui	Alternativa a Visual Studio
Android	Emulador API 34 o dispositivo con depuración USB	SDK de Android instalado por el workload
Git	2.30 o superior	—

3.3 Paquetes NuGet

Paquete	Versión	Proyecto	Uso
Microsoft.Maui.Controls	$(MauiVersion)	App	Marco de interfaz.
Microsoft.Extensions.Logging.Debug	9.0.0	App	Registro en depuración.
CommunityToolkit.Mvvm	8.3.2	App	[ObservableProperty], [RelayCommand].
sqlite-net-pcl	1.9.172	App	ORM SQLite.
SQLitePCLRaw.bundle_green	2.1.10	App	Motor SQLite nativo.
Microsoft.NET.Test.Sdk	17.11.1	Pruebas	Ejecución de pruebas.
xunit	2.9.2	Pruebas	Marco de pruebas.
xunit.runner.visualstudio	2.8.2	Pruebas	Explorador de pruebas de VS.

3.4 Servicios externos

Servicio	Requisito
Google Gemini	Clave de Google AI Studio y un modelo vigente (por defecto gemini-3.5-flash-lite).
Pollinations	Clave de enter.pollinations.ai y un modelo de texto vigente (por defecto openai).
Red	Internet con salida HTTPS (443) a generativelanguage.googleapis.com y gen.pollinations.ai.

4. Requisitos y análisis

4.1 Historias de usuario y criterios de aceptación

ID	Historia de usuario	Criterios de aceptación
HU01	Como estudiante quiero escribir una pregunta y recibir una respuesta de la IA para resolver una duda.	Dado una clave válida y conexión, cuando pulso *Consultar* con 5 a 2000 caracteres, entonces veo el indicador de carga y luego la respuesta; la interfaz no se bloquea.
HU02	Como estudiante quiero elegir una plantilla de prompt para formular mejor mi consulta.	Al elegir una plantilla su texto aparece en el editor y se puede modificar antes de enviar.
HU03	Como estudiante quiero que la app me avise si mi consulta es inválida.	Consulta vacía o de menos de 5 caracteres: no se envía y aparece un mensaje. El contador marca máximo 2000.
HU04	Como estudiante quiero cancelar una consulta lenta.	Al pulsar *Cancelar* aparece «Consulta cancelada.» y puedo consultar de nuevo.
HU05	Como estudiante quiero guardar la consulta y su respuesta para volver a verlas.	*Guardar* se habilita cuando hay respuesta; el registro aparece en el historial y sigue disponible tras reiniciar.
HU06	Como estudiante quiero buscar y filtrar mi historial.	La búsqueda filtra por texto, categoría o proyecto y el interruptor «Solo verificadas» muestra únicamente las verificadas.
HU07	Como estudiante quiero eliminar consultas que ya no necesito.	Se pide confirmación y la lista se actualiza sin recargar la aplicación.
HU08	Como estudiante quiero marcar una respuesta como verificada y anotar de dónde la verifiqué.	El detalle alterna el estado y guarda una nota de hasta 500 caracteres; los cambios persisten.
HU09	Como estudiante quiero agrupar mis consultas por materia o trabajo.	Puedo crear, editar y eliminar proyectos; al eliminar un proyecto sus consultas se conservan.
HU10	Como estudiante quiero consultar dentro de un proyecto para que la IA tenga contexto.	*Consultar en este proyecto* abre Consultar con el proyecto seleccionado y la consulta queda asociada.
HU11	Como estudiante quiero ver mi perfil y mis estadísticas.	El perfil muestra iniciales, consultas, verificadas y proyectos; los datos se guardan solo en el dispositivo.

ID	Historia de usuario	Criterios de aceptación
HU12	Como estudiante quiero registrar mi clave de API sin que se muestre en pantalla y proteger mi privacidad.	La clave se escribe en un campo oculto, se guarda cifrada y solo se informa que existe; si la consulta contiene datos sensibles la app pide confirmación; puedo borrar mis datos.
HU13	Como estudiante quiero elegir el proveedor de IA (Gemini o Pollinations).	En Configuración elijo el proveedor, su clave y su modelo; al guardar, las consultas usan ese proveedor y cada uno conserva su clave.
HU14	Como estudiante quiero que la app no consuma mi cuota sin querer.	A la novena consulta en un minuto aparece «Límite de uso alcanzado: espere N s».
HU15	Como estudiante quiero conocer la finalidad, la privacidad y los límites de la IA.	Aviso de uso responsable al primer inicio y sección «Acerca de» en Configuración.

4.2 Requisitos funcionales

Código	Requisito	Implementación	Estado
RF01	Ingresar una pregunta o instrucción	ChatPage (editor con contador)	Implementado
RF02	Validar contenido y longitud (5–2000)	PromptValidator, ChatViewModel, AiService	Implementado
RF03	Enviar la consulta por POST con cuerpo JSON	AiService.AskAsync	Implementado
RF04	Mostrar estados: carga, respuesta, error, sin conexión, cancelado	ChatViewModel.IsBusy/ StatusMessage, Connectivity, AiErrorMapper	Implementado
RF05	Guardar consulta y respuesta en SQLite	DatabaseService.SaveAsync	Implementado
RF06	Consultar, buscar, filtrar, ver detalle y eliminar del historial	HistoryPage, DetailPage	Implementado
RF07	Plantillas de prompt editables (6)	PromptTemplate.Defaults	Implementado
RF08	Registrar, actualizar y eliminar la clave sin mostrarla, una por proveedor	ApiKeyService (SecureStorage), Entry IsPassword	Implementado
RF09	Sección «Acerca de»: finalidad, autoría, fuentes, privacidad y límites	SettingsPage	Implementado

Código	Requisito	Implementación	Estado
RF10	Ejecutar en Windows y Android	net9.0-windows10.0.19041.0, net9.0-android	Implementado
RF11	Crear, editar, listar y eliminar proyectos	ProjectsPage, ProjectDetailPage	Implementado
RF12	Asociar una consulta a un proyecto y enviar su contexto a la IA	ChatPage, AiService.BuildSystemPrompt	Implementado
RF13	Perfil del estudiante con estadísticas de uso	ProfilePage, ProfileService	Implementado
RF14	Nota personal y marca de verificada por consulta	DetailViewModel	Implementado
RF15	Avisar antes de enviar datos sensibles	SensitiveDataDetector	Implementado
RF16	Limitar la frecuencia (8/min) y la longitud de la respuesta (4096 tokens)	RequestLimiter, MaxOutputTokens	Implementado
RF17	Borrar el historial o todos los datos locales	ProfileViewModel	Implementado
RF18	Elegir manualmente el proveedor de IA: Google Gemini o Pollinations	AiProviderCatalog, AiService, SettingsViewModel	Implementado
RF19	Probar la conexión con el proveedor elegido	SettingsViewModel.TestConnectionAsync	Implementado
RF20	Aviso de uso responsable al primer inicio	HomeViewModel	Implementado

4.3 Requisitos no funcionales

Código	Atributo	Requisito y cómo se atiende
RNF01	Seguridad	Ninguna clave en el código ni en el repositorio; la clave viaja en un encabezado HTTPS (no en la URL) y se guarda en SecureStorage.
RNF02	Privacidad	Aviso visible de no ingresar datos personales; historial y perfil solo en el dispositivo; sin copia de seguridad automática en Android (allowBackup=false).
RNF03	Accesibilidad	SemanticProperties y niveles de encabezado; tema oscuro con contraste ≥ 4.5:1; objetivos táctiles ≥ 46 px.
RNF04	Rendimiento y respuesta	Operaciones asíncronas con tiempo máximo de 60 s y cancelación; la interfaz no se bloquea.
RNF05	Usabilidad	Navegación por pestañas, mensajes claros y confirmación antes

Código	Atributo	Requisito y cómo se atiende
		de eliminar.
RNF06	Mantenibilidad	MVVM con inyección de dependencias; proveedores de IA aislados en AiProviderCatalog y AiService; lógica pura con pruebas.
RNF07	Portabilidad	Un solo proyecto para Windows y Android; diseño adaptable con ancho máximo y ScrollView.
RNF08	Uso responsable	La IA se presenta como apoyo falible: avisos, verificación manual, límites de uso.

Contraste de colores (WCAG 2.1, paleta «Grafito»)

Par (texto / fondo)	Contraste	Mínimo AA
Texto principal / fondo	17.0:1	4.5:1
Texto principal / tarjeta	15.5:1	4.5:1
Texto atenuado / tarjeta	6.6:1	4.5:1
Texto de botón principal / botón	14.5:1	4.5:1
Blanco / botón de éxito	5.1:1	4.5:1
Blanco / botón de peligro	5.8:1	4.5:1
Aviso / fondo de aviso	9.9:1	4.5:1

4.4 Funcionalidades implementadas

Módulo	Funcionalidades	Pantalla
Consulta	Plantillas (Pregunta libre, Explicar un concepto, Resumir, Comparar, Preguntas de práctica, Revisar código), proyecto opcional, contador 0/2000, Consultar, Cancelar, Guardar, aviso de respuesta recortada	Consultar
Proveedores	Google Gemini y Pollinations, modelo por proveedor, clave por proveedor, prueba de conexión	Configuración
Historial	Lista con fecha, categoría, proyecto y verificación; búsqueda; «Solo verificadas»; eliminar	Historial, Detalle
Detalle	Respuesta completa, verificar/quitar verificación, nota personal, eliminar	Detalle de consulta
Proyectos	Crear, editar, eliminar (conserva consultas), contador de consultas, consultar en el proyecto	Proyectos, Detalle de proyecto
Perfil	Nombre, programa, institución, iniciales, estadísticas, borrar historial, borrar todos los datos	Perfil
Seguridad y privacidad	SecureStorage, campo oculto, detector de datos sensibles, límite de 8 consultas/min, aviso de uso responsable	Varias
Errores	Mensajes para validación, sin red, sin clave, límite de uso,	Consultar,

Módulo	Funcionalidades	Pantalla
	400, 401, 402, 403, 404, 408, 429, 5xx, tiempo agotado y cancelación	Configuración

5. Arquitectura del sistema

5.1 Diagrama de arquitectura

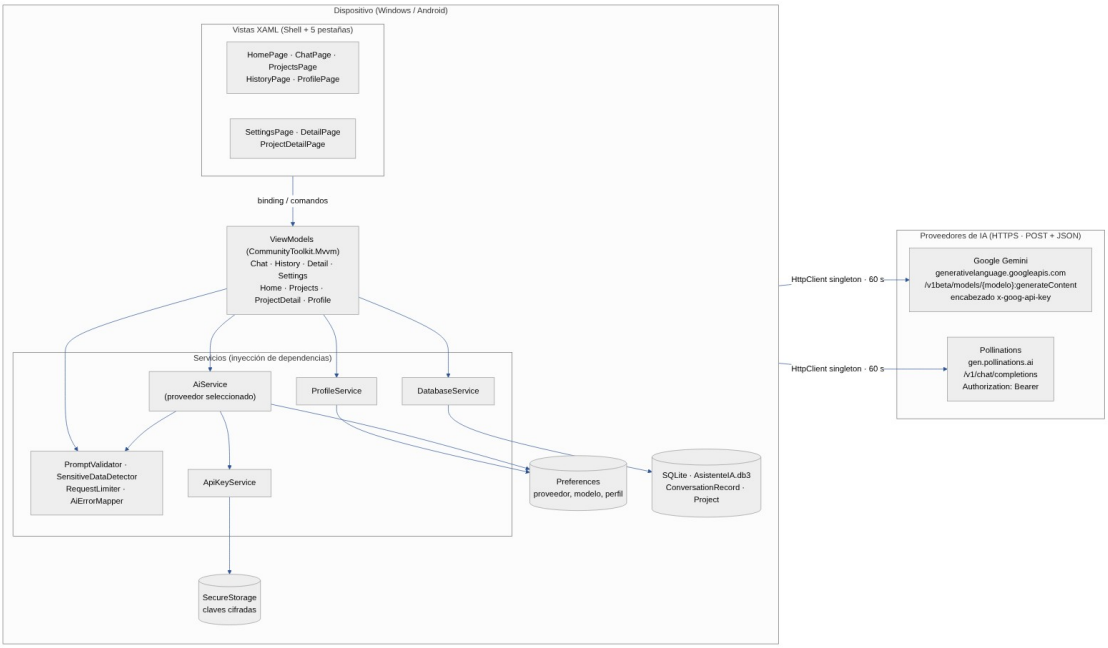


Figura 1. Arquitectura del sistema (fuente editable: diagramas/fuentes/arquitectura.mmd).

5.2 Componentes

Capa	Componentes	Responsabilidad	Se comunica con
Vistas (XAML + Shell)	5 pestañas: Inicio, Consultar, Proyectos, Historial, Perfil. Rutas secundarias: Configuración, Detalle, Detalle de proyecto.	Mostrar datos y capturar acciones. No hacen solicitudes HTTP.	ViewModels por *binding* y comandos.
ViewModels (MVVM)	HomeViewModel, ChatViewModel, ProjectsViewModel, ProjectDetailViewModel, HistoryViewModel, DetailViewModel, ProfileViewModel, SettingsViewModel.	Estado observable, validación, comandos.	Servicios inyectados por constructor.
Servicios	AiService, ApiKeyService, DatabaseService, ProfileService, INavigationService, IDialogService.	Acceso a la API de IA, almacenamiento seguro, SQLite, preferencias, navegación.	APIs externas y almacenes locales.

Capa	Componentes	Responsabilidad	Se comunica con
Lógica pura	PromptValidator, SensitiveDataDetector, RequestLimiter, AiErrorMapper, AiProviderCatalog.	Reglas sin dependencia de MAUI; probadas con xUnit.	—
Almacenamiento local	SQLite (AsistenteIA.db3), SecureStorage, Preferences.	Historial y proyectos; claves cifradas; proveedor, modelo y perfil.	Solo el dispositivo.
Proveedores de IA	Google Gemini · Pollinations.	Generar la respuesta.	HTTPS, POST + JSON, HttpClient único (60 s).

5.3 Comunicación entre componentes

1. **Vista → ViewModel:** *data binding* ({Binding}) y comandos generados con [RelayCommand].
2. **ViewModel → AiService:** AskAsync(prompt, …, CancellationToken); asíncrono, no bloquea la interfaz.
3. **AiService → proveedor:** según el proveedor guardado en Preferences arma el POST de Gemini (generateContent) o de Pollinations (chat/completions). La clave se lee de SecureStorage y viaja en un encabezado.
4. **ViewModel → DatabaseService → SQLite:** al pulsar *Guardar*, *Verificar*, *Nota* o *Eliminar*.
5. **Inyección de dependencias** (MauiProgram.cs): servicios *singleton*, ViewModels y páginas *transient*.

5.4 Estructura de la solución

Carpeta	Contenido	Responsabilidad
Models	ChatModels (JSON de Gemini y de Pollinations), ConversationRecord, Project (entidades SQLite), PromptTemplate, UserProfile, AiProvider	Representar los datos.
Services	AiService, AiErrorMapper, ApiKeyService, DatabaseService, ProfileService, PromptValidator, SensitiveDataDetector, RequestLimiter, INavigationService, IDialogService	Acceso a la API REST, almacenamiento seguro, base de datos y reglas.
ViewModels	HomeViewModel, ChatViewModel, ProjectsViewModel, ProjectDetailViewModel, HistoryViewModel, DetailViewModel, ProfileViewModel, SettingsViewModel	Estado, comandos y reglas de presentación.
Views	HomePage, ChatPage, ProjectsPage,	Interfaz XAML; solo enlaza

Carpeta	Contenido	Responsabilidad
	ProjectDetailPage, HistoryPage, DetailPage, ProfilePage, SettingsPage	datos y comandos.
Resources	Colors.xaml, Styles.xaml, fuentes e imágenes	Estilos y tema visual.

5.5 Recorrido de una consulta

1. **XAML (ChatPage):** el usuario escribe el prompt y pulsa *Consultar*; el editor y los botones están enlazados al ViewModel.
2. **ViewModel (ChatViewModel):** AskCommand valida el texto (PromptValidator), revisa datos sensibles (SensitiveDataDetector), activa el estado de carga y crea un CancellationTokenSource.
3. **Servicio (AiService):** comprueba la conectividad, lee el proveedor elegido y su clave de SecureStorage, aplica el límite de uso y envía el POST con HttpClient.
4. **JSON:** el cuerpo y la respuesta se serializan con las clases de ChatModels (formato Gemini o formato OpenAI para Pollinations).
5. **ViewModel:** muestra la respuesta o el mensaje de error (AiErrorMapper) y habilita *Guardar*.
6. **SQLite (DatabaseService):** al pulsar *Guardar* se inserta un ConversationRecord; HistoryViewModel lo carga en una ObservableCollection.

5.6 Inyección de dependencias y navegación

En MauiProgram.cs se registra un único HttpClient reutilizable (dirección base de Gemini y tiempo máximo de 60 s; Pollinations usa URL absoluta), los servicios como *singleton* y los ViewModels y páginas como *transient*. Las páginas reciben su ViewModel por constructor y lo asignan como BindingContext.

AppShell define una barra de pestañas (Inicio, Consultar, Proyectos, Historial y Perfil). Configuración, Detalle de consulta y Detalle de proyecto son rutas secundarias registradas con Routing.RegisterRoute y se abren con GoToAsync, por ejemplo GoToAsync(nameof(DetailPage), parámetros).

Pantalla	Tipo	Función
Inicio	Pestaña	Saludo, accesos rápidos, recomendaciones; aviso de uso responsable la primera vez
Consultar	Pestaña	Plantilla, proyecto, consulta, respuesta, guardar y cancelar
Proyectos	Pestaña	Lista de proyectos y botón para crear
Historial	Pestaña	Búsqueda, filtro de verificadas, eliminar
Perfil	Pestaña	Datos personales, estadísticas, privacidad
Detalle de consulta	Secundaria	Respuesta completa, nota, verificar, eliminar
Detalle de	Secundaria	Crear, editar y eliminar proyecto; sus consultas

Pantalla	Tipo	Función
proyecto		
Configuración y Acerca de	Secundaria	Proveedor, clave, modelo, probar conexión, autoría, privacidad, límites

5.7 Decisiones técnicas

Decisión	Alternativa	Justificación
.NET MAUI + Shell	Xamarin, WPF	Un proyecto para Windows y Android; Shell da pestañas y rutas.
Dos proveedores de IA (Gemini y Pollinations) con cambio manual	Un solo proveedor	La guía propone Pollinations y Gemini tiene una API bien documentada; aislar los proveedores en AiProviderCatalog/AiService permite cambiar sin tocar las vistas y el usuario siempre sabe a quién envía su texto.
Clave en SecureStorage y en un encabezado HTTP	Clave en el código, en un archivo o en la URL	Evita exponer la credencial en el repositorio, en capturas y en registros de red.
Una clave y un modelo por proveedor	Una sola clave	Se puede alternar de proveedor sin volver a escribirlos.
Modelo configurable por el usuario	Modelo fijo en el código	El proveedor cambia modelos y cuotas sin necesidad de recompilar.
MVVM con CommunityToolkit.Mvvm	Lógica en el *code-behind*	Separación clara, menos código repetitivo y lógica probable con pruebas unitarias.
INavigationService / IDialogService	Shell.Current en el ViewModel	Los ViewModels no dependen de la interfaz.
SQLite con sqlite-net-pcl	Archivos JSON o Preferences; EF Core	Consultas y CRUD sobre tablas con clave primaria autoincremental; ligero y asíncrono.
Un único HttpClient *singleton*	Crear uno por solicitud	Reutiliza conexiones y evita el agotamiento de sockets.
Cancelación con CancellationToken y tiempo máximo	Esperar sin límite	El usuario puede cancelar y la espera termina con un mensaje.
Límite de uso en el cliente	Confiar solo en el proveedor	Protege la cuota de la clave del estudiante.
Detección de datos sensibles (heurística)	Bloquear siempre	Avisa y deja decidir al usuario.
Lógica pura sin MAUI	Lógica dentro de los ViewModels	Se prueba con xUnit sin emulador.

6. Diagrama de clases

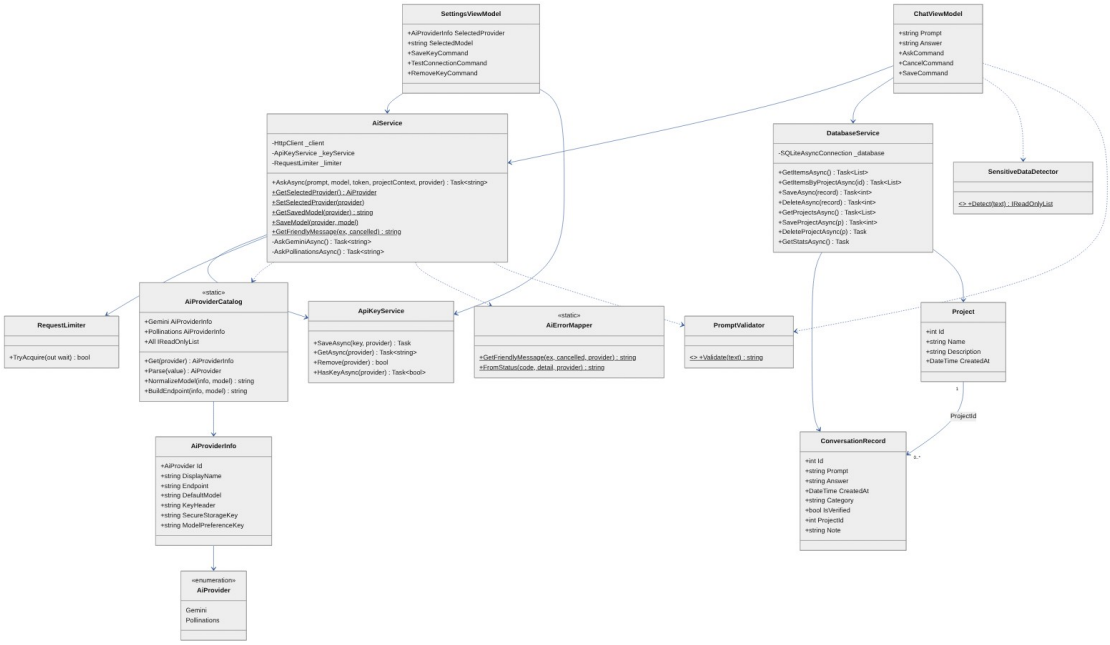


Figura 2. Diagrama de clases (fuente editable: diagramas/fuentes/clases.mmd).

6.1 Clases principales

Clase	Tipo	Atributos / propiedades clave	Métodos clave	Responsabilidad
AiService	Servicio (singleton)	_client (HttpClient), _keyService, _limiter; constantes MaxOutputTokens = 4096, MaxPromptLength = 2000	AskAsync, GetSelectedProvider, SetSelectedProvider, GetSavedModel, SaveModel, GetFriendlyMessage	Validar, comprobar red/clave/límite y enviar la consulta al proveedor elegido.
AiProviderCatalog / AiProviderInfo / AiProvider	Lógica pura	Endpoint, modelo por defecto, modelos sugeridos, encabezado de clave, nombre en SecureStorage y Preferences	Get, Parse, NormalizeModel, BuildEndpoint	Describir cada proveedor en un solo lugar.
AiErrorMapper	Lógica pura	—	GetFriendlyMessa	Convertir

Clase	Tipo	Atributos / propiedades clave	Métodos clave	Responsabilidad
	(estática)		ge, FromStatus	excepciones y códigos HTTP en mensajes en español sin la clave.
ApiKeyService	Servicio	—	SaveAsync, GetAsync, Remove, HasKeyAsync (por proveedor)	Guardar la clave en SecureStorage.
DatabaseService	Servicio	_database (SQLiteAsyncConnection, inicialización perezosa)	CRUD de ConversationRecord y Project, GetStatsAsync	Persistencia local.
ProfileService	Servicio	Claves de Preferences	Load, Save, Clear, NoticeAccepted	Perfil del estudiante y aviso de uso responsable.
PromptValidator	Lógica pura	MinLength = 5, MaxLength = 2000	Validate	RF02.
SensitiveDataDetector	Lógica pura	Expresiones regulares con *timeout* de 200 ms	Detect, PassesLuhn	Avisar de correos, tarjetas, teléfonos, credenciales y claves.
RequestLimiter	Lógica	Máx. 8 solicitudes por minuto	TryAcquire(out wait)	Proteger la cuota de la clave.
ConversationRecord, Project	Entidades SQLite	Ver diccionario de datos	Propiedades de presentación con [Ignore]	Datos persistentes.
ChatModels	DTO JSON	ChatRequest/ ChatResponse (Gemini), OpenAiChatRequest/OpenAiChatResponse (Pollinations)	—	Serialización con System.Text.Json.
ViewModels	ObservableObject	[ObservableProperty]	[RelayCommand]	Estado y comandos de cada pantalla.

6.2 Relaciones

- **Composición por inyección:** ChatViewModel → AiService, DatabaseService, IDialogService; SettingsViewModel → ApiKeyService, AiService; AiService → HttpClient, ApiKeyService, RequestLimiter.
- **Dependencias de uso:** AiService usa AiProviderCatalog, PromptValidator y AiErrorMapper; ChatViewModel usa SensitiveDataDetector.
- **Asociación de datos:** un Project agrupa 0..* ConversationRecord mediante ProjectId (0 = sin proyecto).

7. Arquitectura de proveedores de IA

La aplicación soporta **Google Gemini** y **Pollinations**; el usuario cambia de proveedor **manualmente** en Configuración.

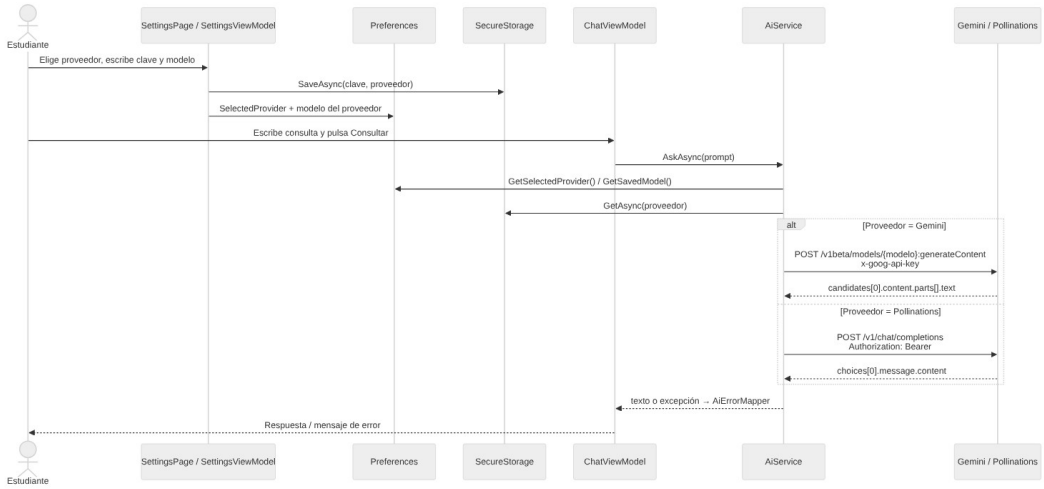


Figura 3. Selección del proveedor y envío de la consulta.

7.1 Diseño

Pieza	Archivo	Función
AiProvider (enum)	Models/ AiProvider.cs	Gemini = 0, Pollinations = 1.
AiProviderInfo	Models/ AiProvider.cs	Datos de cada proveedor: nombre, endpoint, modelo por defecto, modelos sugeridos, encabezado de clave, clave en SecureStorage y preferencia del modelo.
AiProviderCatalog	Models/ AiProvider.cs	Lista de proveedores, Parse (valor guardado → enum, con Gemini como respaldo), NormalizeModel, BuildEndpoint.
AiService	Services/ AiService.cs	AskAsync hace las validaciones comunes y llama a AskGeminiAsync o AskPollinationsAsync.
ChatModels	Models/ ChatModels.cs	DTO de Gemini (ChatRequest/ChatResponse) y de OpenAI/Pollinations (OpenAiChatRequest/OpenAiChatResponse).
SettingsViewModel / SettingsPage	ViewModels, Views	Selector de proveedor, modelos, clave, *Probar conexión*.

7.2 Características de cada proveedor

Aspecto	Google Gemini	Pollinations
Endpoint	/v1beta/models/	/v1/chat/completions

Aspecto	Google Gemini	Pollinations
	{modelo}:generateContent	
Formato	Propio de Google (contents, parts)	Compatible con OpenAI (messages)
Autenticación	x-goog-api-key	Authorization: Bearer
Modelo por defecto	gemini-3.5-flash-lite	openai
Modelos sugeridos	gemini-3.5-flash-lite, gemini-3.1-flash-lite, gemini-3.6-flash, gemini-3.8-flash	openai, openai-fast, mistral (lista vigente en gen.pollinations.ai/models)
Obtener clave	Google AI Studio	enter.pollinations.ai
Respuesta	candidates[0].content.parts[].text	choices[0].message.content

7.3 Selección y cambio manual

1. En **Perfil › Configuración**, el usuario elige el proveedor en el selector «Proveedor de IA».
2. Al cambiar el selector se cargan los modelos sugeridos y el estado de la clave **de ese proveedor**; aún no se usa en las consultas.
3. Al pulsar **Guardar** se guarda la clave (si se escribió una), el modelo en su preferencia (SelectedModel o SelectedModel_Pollinations) y el proveedor en SelectedProvider.
4. **Probar conexión** prueba el proveedor del selector aunque no se haya guardado.
5. Desde ese momento ChatViewModel muestra «Consultando {proveedor}…» y AiService usa el proveedor guardado. Cada proveedor conserva su propia clave y modelo, por lo que se puede alternar sin volver a escribirlos.

No hay cambio automático (*fallback*): si un proveedor falla, el mensaje de error sugiere cambiarlo manualmente. Así el usuario siempre sabe a qué servicio se envía su texto.

7.4 Cómo agregar otro proveedor

1. Agregar el valor al enum AiProvider y una entrada AiProviderInfo en AiProviderCatalog (endpoint, encabezado, claves).
2. Si usa formato OpenAI, reutilizar AskPollinationsAsync (o generalizarlo); si no, crear AskXxxAsync y sus DTO.
3. Agregar el caso en el switch de AiService.AskAsync.
4. Agregar pruebas en AiProviderCatalogTests. Las vistas no cambian.

8. Interfaz de la aplicación

La aplicación usa un tema oscuro («Grafito») con tarjetas y navegación por pestañas. Las capturas corresponden a la ejecución en Windows.

8.1 Pantalla de inicio

Presenta el propósito de la aplicación, accesos rápidos a las demás pantallas y recomendaciones de uso, incluida la advertencia de verificar las respuestas y no escribir datos personales. La primera vez muestra el aviso «Uso responsable de la IA».

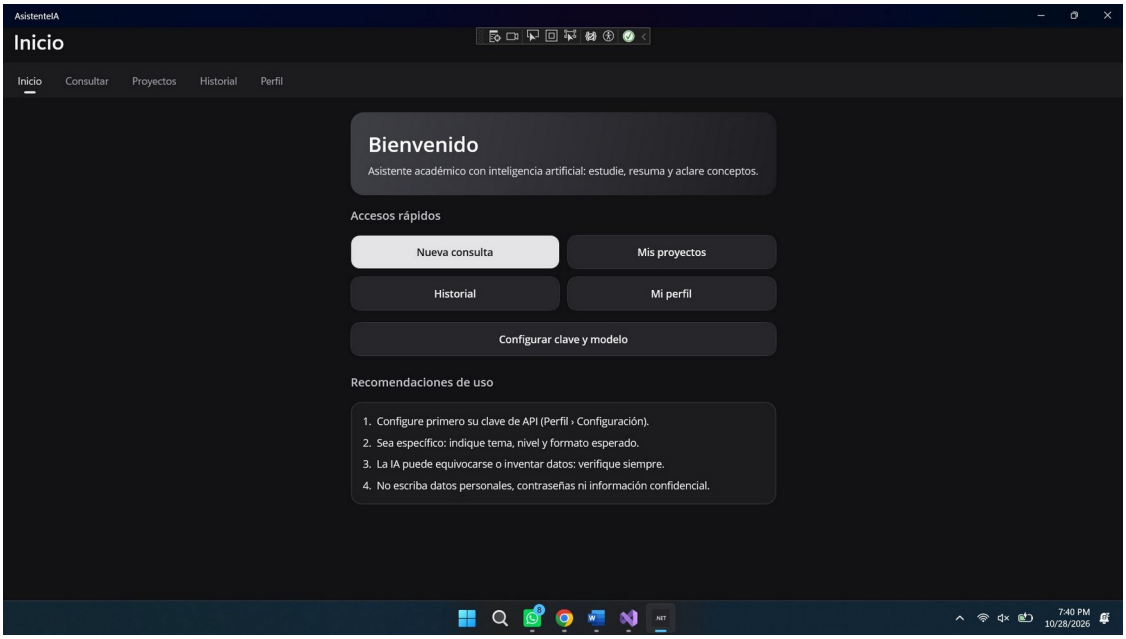


Figura 4. Pantalla de inicio.

8.2 Pantalla de consulta

Permite elegir una plantilla de prompt y, opcionalmente, un proyecto; captura la consulta (con contador de 2000 caracteres) y muestra la respuesta. Incluye los botones Consultar, Cancelar y Guardar. Mientras consulta muestra «Consultando {proveedor}…».

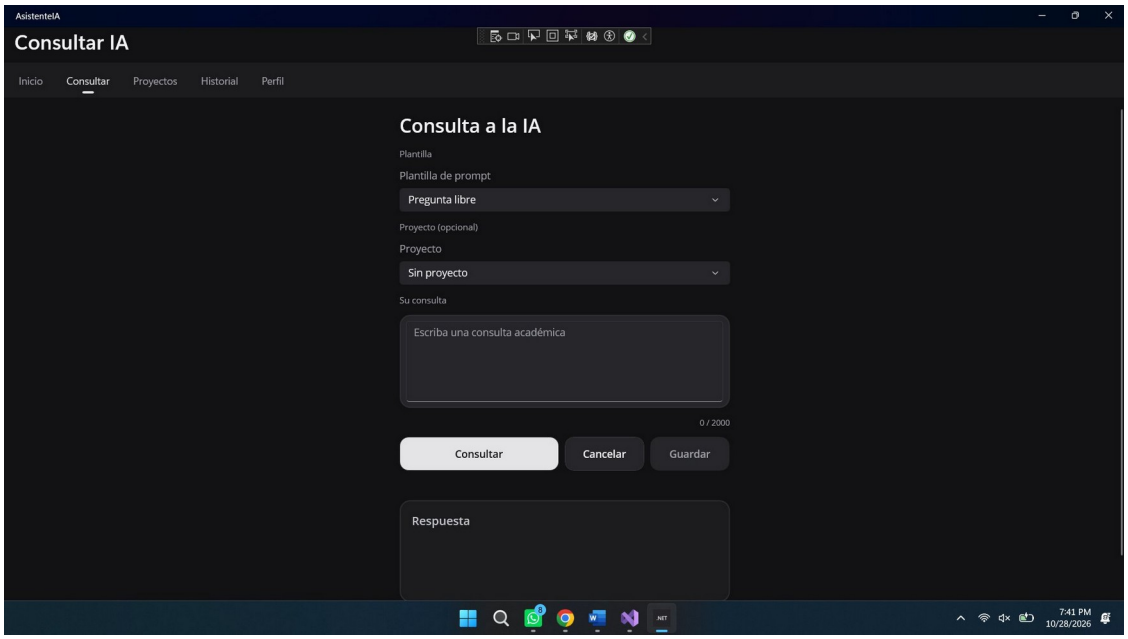


Figura 5. Pantalla de consulta.

8.3 Pantalla de proyectos

Agrupa las consultas por materia, taller o trabajo. Cuando no hay proyectos muestra un mensaje que invita a crear el primero. Al abrir un proyecto se muestra el **Detalle de proyecto**: nombre (3–60 caracteres), descripción (máx. 300), sus consultas y los botones *Guardar proyecto*, *Consultar en este proyecto* y *Eliminar proyecto*.

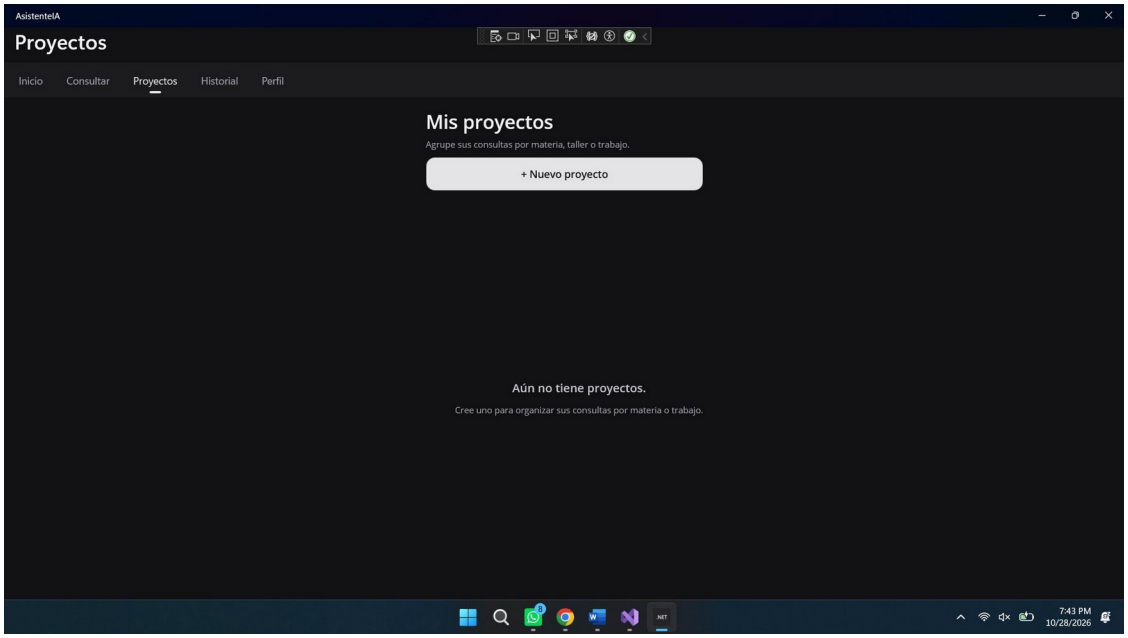


Figura 6. Pantalla de proyectos.

8.4 Pantalla de historial

Lista las consultas guardadas con fecha, categoría, proyecto y estado de verificación. Ofrece búsqueda, filtro «Solo verificadas» y eliminación. Al tocar una consulta se abre el **Detalle de consulta**, donde se puede marcar como verificada, escribir una nota personal (máx. 500) y eliminarla.

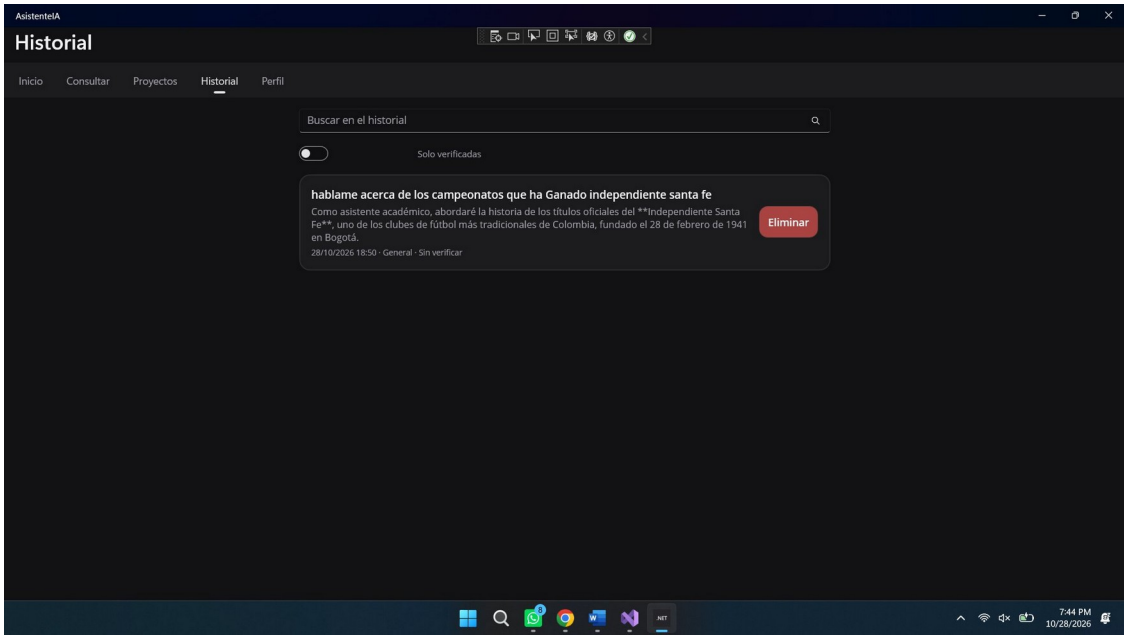


Figura 7. Pantalla de historial.

8.5 Pantalla de perfil de usuario

Muestra contadores de consultas, verificadas y proyectos, y permite guardar nombre, programa e institución. Estos datos se almacenan solo en el dispositivo y no se envían a la IA. Incluye *Borrar historial de consultas* y *Borrar todos mis datos locales*.

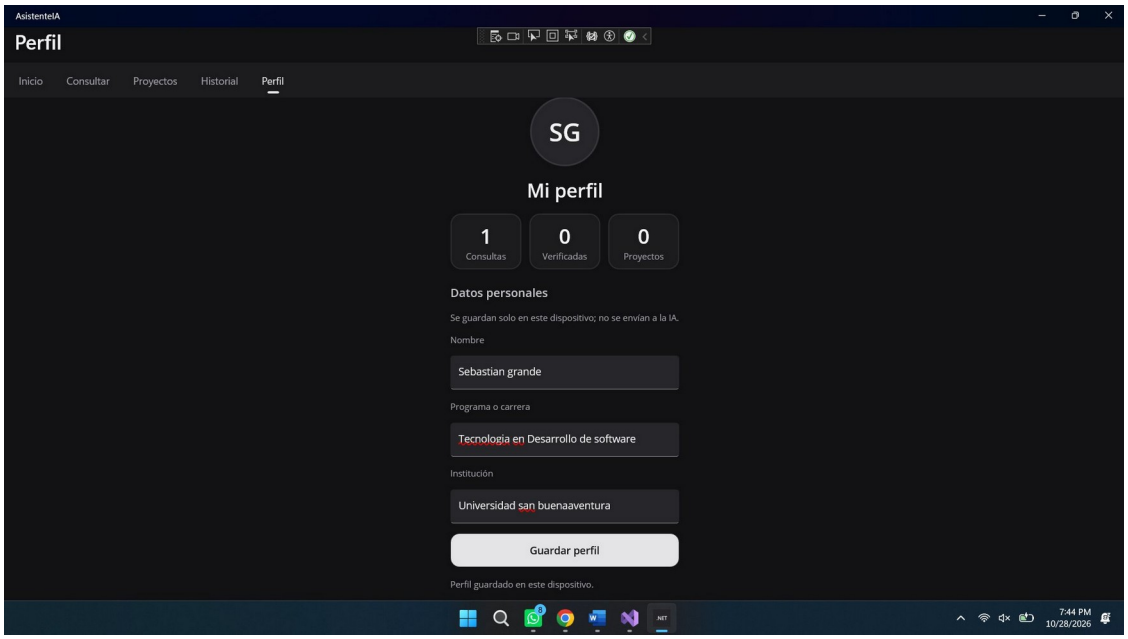


Figura 8. Pantalla de perfil de usuario.

8.6 Pantalla de configuración de la API

Permite elegir el **proveedor de IA** (Google Gemini o Pollinations), registrar la clave en un campo oculto, elegir o escribir el modelo, probar la conexión y eliminar la clave. La línea «Proveedor activo» indica qué proveedor y modelo usan las consultas. Incluye el aviso de privacidad y la sección «Acerca de» (finalidad, autoría, fuentes, privacidad y límites). La clave no se muestra en pantalla.

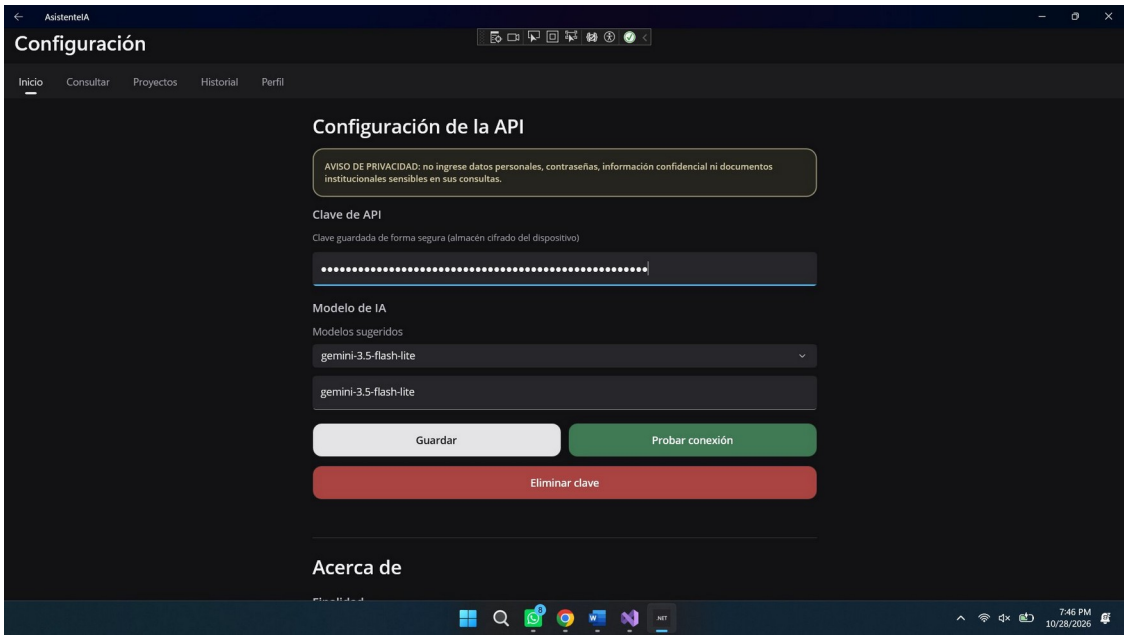


Figura 9. Pantalla de configuración de la API (clave oculta). El selector «Proveedor de IA» se ubica sobre el campo de la clave.

8.7 Conexión de la clave de API

El botón «Probar conexión» realiza una consulta breve al proveedor seleccionado y muestra el resultado.

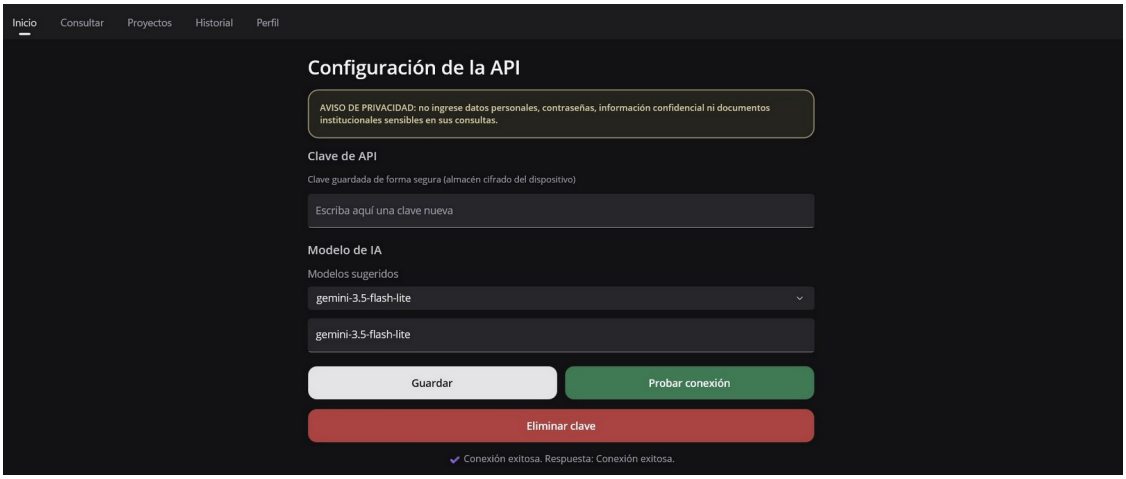


Figura 10. Conexión de la API exitosa desde la pantalla de configuración.

9. Documentación de la API REST

La aplicación **consume** dos API REST de terceros (no expone una API propia). Ambas se llaman con un único HttpClient registrado como *singleton* (tiempo máximo **60 s**), método **POST**, cuerpo **JSON** y la clave **en un encabezado**, nunca en la URL.

9.1 Google Gemini — generateContent

Elemento	Valor
Método y URL	POST https://generativelanguage.googleapis.com/v1beta/models/{modelo}:generateContent
Parámetro de ruta	{modelo}: identificador del modelo (por defecto gemini-3.5-flash-lite). Si el usuario escribe models/x, se limpia a x.
Encabezados	x-goog-api-key: <clave> · Content-Type: application/json; charset=utf-8

Solicitud

```
{
  "systemInstruction": { "parts": [ { "text": "Eres un asistente académico. Responde de forma clara, indica la incertidumbre cuando exista y no inventes fuentes ni citas." } ] },
  "contents": [ { "role": "user", "parts": [ { "text": "Explica qué es una API REST" } ] } ],
  "generationConfig": { "maxOutputTokens": 4096 }
}
```

Campo	Obligatorio	Descripción
systemInstruction.parts[].text	No	Instrucción de sistema. Si la consulta tiene proyecto, se agrega «Contexto del proyecto…» (máx. 400 caracteres).
contents[].role / parts[].text	Sí	Turno del usuario con la consulta (5–2000 caracteres).
generationConfig.maxOutputTokens	No	Límite de longitud de la respuesta (4096).

Respuesta 200

```
{
  "candidates": [ {
    "content": { "parts": [ { "text": "Una API REST es …" } ] },
    "finishReason": "STOP"
  } ],
  "promptFeedback": { "blockReason": null }
}
```

Campo	Uso en la app
candidates[0].content.parts[].text	Se concatenan y se muestran.
candidates[0].finishReason = "MAX_TOKENS"	Se agrega el aviso «Respuesta recortada por el límite de longitud».
promptFeedback.blockReason	Si no hay texto: «El servicio bloqueó la consulta (motivo)».

Respuesta de error

{ "error": { "code": 400, "message": "API key not valid. Please pass a valid API key.", "status": "INVALID_ARGUMENT" } }

9.2 Pollinations — chat/completions (compatible con OpenAI)

Elemento	Valor
Método y URL	POST https://gen.pollinations.ai/v1/chat/completions
Encabezados	Authorization: Bearer <clave> · Content-Type: application/json; charset=utf-8
Clave	Se crea en https://enter.pollinations.ai. La API pública de texto ya no permite uso anónimo sin límites.

Solicitud

```
{
 "model": "openai",
 "messages": [
  { "role": "system", "content": "Eres un asistente académico. ..." },
  { "role": "user", "content": "Explica qué es una API REST" }
 ],
 "max_tokens": 4096
}
```

Respuesta 200

{
 "choices": [{ "message": { "role": "assistant", "content": "Una API REST es ..." }, "finish_reason": "stop"
}]
}

Campo	Uso en la app
choices[0].message.content	Texto mostrado.
choices[0].finish_reason = "length"	Aviso de respuesta recortada.
Error	Se lee {"error":{"message":...}}, {"error":"..."} o texto plano (máx. 200 caracteres).

9.3 Códigos de estado

Código	Significado	Mensaje en la app
200	Correcto	Se muestra la respuesta y se habilita *Guardar*.
400	Solicitud inválida (en Gemini, también clave inválida)	«Error de autorización (400): la clave de API no es válida» o «Solicitud rechazada (400). Revise el modelo configurado».
401 / 403	Sin autorización / sin permisos	«Error de autorización (401/403): la clave de {proveedor} es inválida o no tiene permisos».
402	Sin saldo o límite gratuito (Pollinations)	«{proveedor} rechazó la solicitud por saldo o límite gratuito (402)…»
404	Modelo no encontrado	«Modelo no encontrado (404). Cambie el modelo en Configuración».
408	El servidor agotó su tiempo	«El servidor tardó demasiado en responder (408)…»
429	Cuota o límite de solicitudes	«Se superó la cuota o el límite de solicitudes (429)…»
5xx	Falla del proveedor	«El servicio de IA no está disponible (código)…»

El detalle completo del manejo de errores está en el capítulo 10.

9.4 Ejemplo real de consulta

La evidencia de la prueba P01 (capítulo 14) muestra la consulta «Explica qué es una API REST» enviada con la plantilla «Pregunta libre» y el proyecto «Pruebas»; la respuesta del proveedor se muestra en la tarjeta *Respuesta* y se habilita *Guardar*.

10. Gestión de errores y excepciones

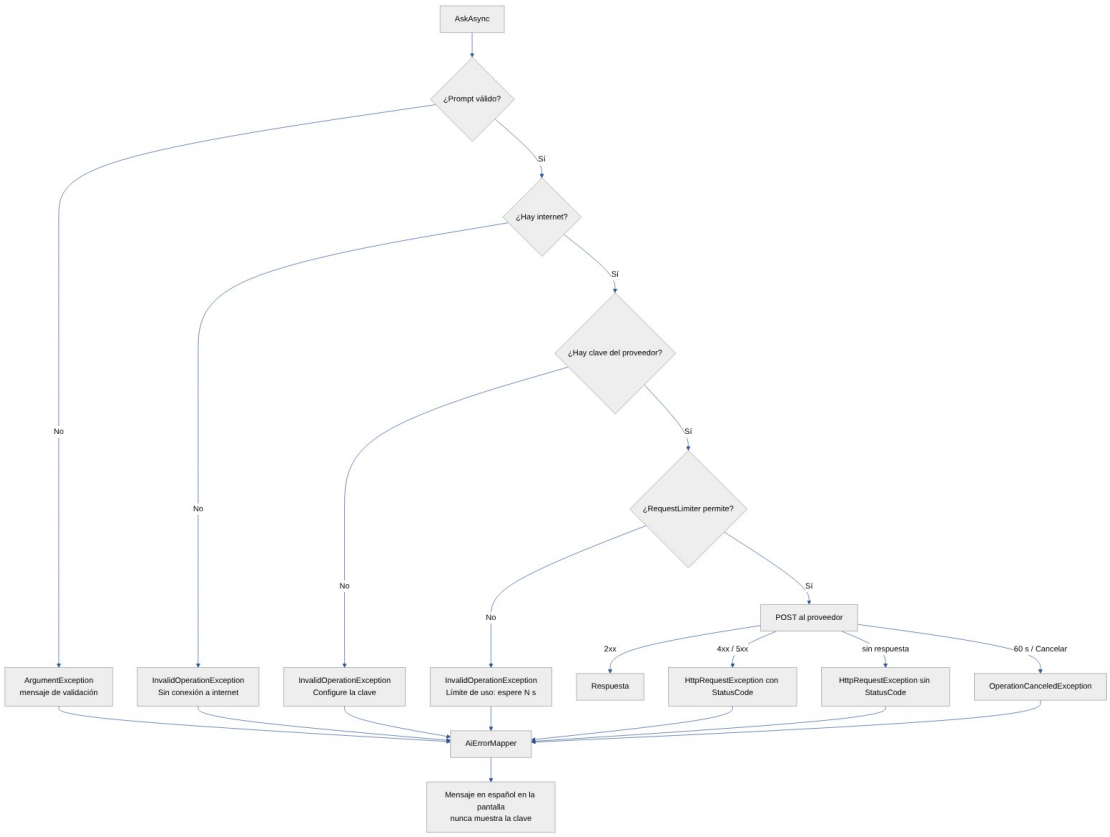


Figura 11. Flujo de manejo de errores.

Todas las excepciones terminan en **AiErrorMapper.GetFriendlyMessage** (lógica pura, probada con 16 pruebas), que devuelve un mensaje en español **sin la clave**. La interfaz nunca se cierra: el ViewModel captura la excepción, muestra el mensaje en StatusMessage y deja los botones activos.

10.1 Errores antes de enviar

Situación	Excepción	Mensaje
Consulta vacía	ArgumentException	«Escriba una consulta antes de enviar.»
Menos de 5 / más de 2000 caracteres	ArgumentException	«La consulta es demasiado corta (mínimo 5 caracteres).» / «La consulta supera 2000 caracteres.»
Sin internet (Connectivity.NetworkAccess)	InvalidOperationException	«Sin conexión a internet. Verifique su red e intente de nuevo.»
Sin clave del proveedor	InvalidOperationException	«Configure la clave de {proveedor} en Perfil ›

Situación	Excepción	Mensaje
		Configuración.»
Más de 8 consultas por minuto	InvalidOperationException	«Límite de uso alcanzado: espere N s…»
Datos sensibles detectados	(diálogo, no excepción)	El usuario decide *Enviar* o *Revisar*.

10.2 Errores HTTP

Código	Causa típica	Mensaje	Acción sugerida
400	Gemini: clave inválida («API key») o cuerpo/modelo inválido	«Error de autorización (400)…» o «Solicitud rechazada (400). Revise el modelo…»	Revisar clave o modelo.
401	Clave ausente o inválida	«Error de autorización (401): la clave de {proveedor} es inválida…»	Guardar una clave nueva.
402	Pollinations sin saldo o límite gratuito	«{proveedor} rechazó la solicitud por saldo o límite gratuito (402)…»	Revisar la cuenta o cambiar de proveedor.
403	Clave sin permisos / API no habilitada	«Error de autorización (403)…»	Revisar permisos del proyecto.
404	Modelo inexistente o retirado	«Modelo no encontrado (404). Cambie el modelo en Configuración.»	Elegir un modelo sugerido.
408	El servidor agotó su tiempo de espera	«El servidor tardó demasiado en responder (408)…»	Reintentar.
429	Cuota o límite por minuto	«Se superó la cuota o el límite de solicitudes (429)…»	Esperar.
500–599	Falla del proveedor	«El servicio de IA no está disponible (código)…»	Reintentar más tarde o cambiar de proveedor.
Otro	—	«Error del servicio: código.»	—

10.3 Problemas de conexión y tiempo

Situación	Excepción	Mensaje
DNS, TLS o servidor inalcanzable	HttpRequestException sin StatusCode	«No se pudo conectar con el servicio. Verifique su conexión.»
Se superan los 60 s del HttpClient	TaskCanceledException / TimeoutException	«Tiempo de espera agotado. Intente de nuevo.»
El usuario pulsa **Cancelar**	OperationCanceledException con el token cancelado	«Consulta cancelada.»

Situación	Excepción	Mensaje
Respuesta 200 sin texto	—	«El servicio no devolvió contenido.» o «El servicio bloqueó la consulta (motivo) …»
Error al guardar en SQLite	Exception	«No se pudo guardar localmente: …»

Las pruebas P02 a P06 del capítulo 14 muestran estos mensajes en ejecución (validación, sin internet, clave inválida, tiempo agotado y cancelación).

11. Persistencia con SQLite

DatabaseService inicializa una SQLiteAsyncConnection en la carpeta de datos de la aplicación (FileSystem.AppDataDirectory) y crea las tablas a partir de las entidades ConversationRecord y Project.

11.1 Modelo entidad-relación

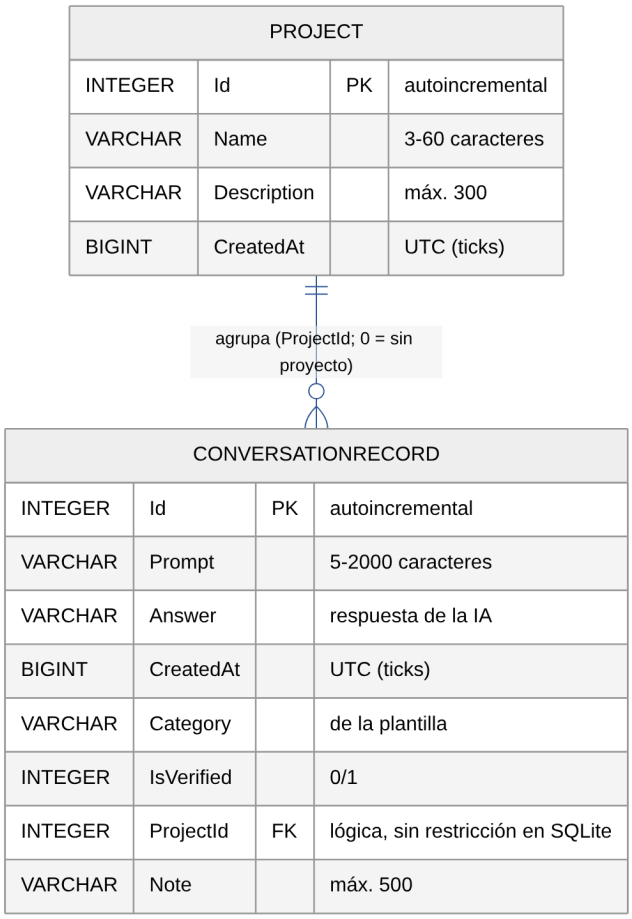


Figura 12. Modelo entidad-relación (fuente editable: diagramas/fuentes/erd.mmd).

- **Base de datos:** archivo AsistenteIA.db3 en FileSystem.AppDataDirectory (carpeta privada de la app).
- **Biblioteca:** sqlite-net-pcl 1.9.172 con SQLitePCLRaw.bundle_green 2.1.10. Las tablas se crean con CreateTableAsync<T>(), que también agrega columnas nuevas a bases existentes.

Tablas

Tabla	Clave primaria	Claves foráneas	Descripción
Project	Id (INTEGER, autoincremental)	—	Materia, taller o trabajo que agrupa consultas.
ConversationRecord	Id (INTEGER, autoincremental)	ProjectId → Project.Id (**lógica**)	Consulta guardada con su respuesta.

Relación

Relación	Cardinalidad	Regla
Project agrupa ConversationRecord	1 : 0..*	ProjectId = 0 significa «sin proyecto».

Integridad referencial: sqlite-net no crea restricciones FOREIGN KEY. La integridad la mantiene DatabaseService.DeleteProjectAsync: antes de borrar un proyecto ejecuta UPDATE ConversationRecord SET ProjectId = 0 WHERE ProjectId = ?, así las consultas se conservan sin proyecto.

SQL equivalente (generado por sqlite-net)

```
CREATE TABLE "Project" (
  "Id" INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL,
  "Name" VARCHAR, "Description" VARCHAR, "CreatedAt" BIGINT);

CREATE TABLE "ConversationRecord" (
  "Id" INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL,
  "Prompt" VARCHAR, "Answer" VARCHAR, "CreatedAt" BIGINT,
  "Category" VARCHAR, "IsVerified" INTEGER, "ProjectId" INTEGER, "Note" VARCHAR);
```

11.2 Diccionario de datos

Las longitudes las valida la aplicación (ViewModels y PromptValidator); en SQLite las columnas VARCHAR no tienen límite. DateTime se guarda como *ticks* (BIGINT), que es el comportamiento por defecto de sqlite-net.

Tabla ConversationRecord (historial)

Campo	Tipo C#	Tipo SQLite	Restricciones	Valor por defecto	Función
Id	int	INTEGER	PK, autoincremental, no nulo	asignado por SQLite	Identificador. 0 = registro nuevo (insertar).
Prompt	string	VARCHAR	5–2000 caracteres (PromptValidator)	""	Consulta enviada a la IA.

Campo	Tipo C#	Tipo SQLite	Restricciones	Valor por defecto	Función
Answer	string	VARCHAR	—	""	Respuesta recibida.
CreatedAt	DateTime	BIGINT	UTC	DateTime.UtcNow	Fecha; se muestra en hora local (dd/MM/yyyy HH:mm).
Category	string	VARCHAR	Categoría de la plantilla	"General"	Explicación, Resumen, Comparación, Estudio, Programación…
IsVerified	bool	INTEGER (0/1)	—	false	El estudiante contrastó la respuesta.
ProjectId	int	INTEGER	FK lógica a Project.Id; 0 = sin proyecto	0	Agrupa la consulta.
Note	string	VARCHAR	máx. 500 caracteres	""	Nota personal editable en el detalle.
ProjectName, CreatedAtLocal, VerificationText, MetaText	string	— ([Ignore])	No se guardan	—	Solo presentación.

Tabla Project

Campo	Tipo C#	Tipo SQLite	Restricciones	Valor por defecto	Función
Id	int	INTEGER	PK, autoincremental	asignado por SQLite	Identificador.
Name	string	VARCHAR	3–60 caracteres, obligatorio	""	Nombre del proyecto.
Description	string	VARCHAR	máx. 300 caracteres	""	Contexto que se envía a la IA (máx. 400 caracteres incluido el nombre).
CreatedAt	DateTime	BIGINT	UTC	DateTime.UtcNow	Fecha de creación.
ConversationCount, CountText,	—	— ([Ignore	No se guardan	—	Presentación.

Campo	Tipo C#	Tipo SQLite	Restricciones	Valor por defecto	Función
DescriptionText		])			

Preferences (preferencias del dispositivo, sin cifrar)

Clave	Tipo	Ejemplo	Función
SelectedProvider	string	Gemini / Pollinations	Proveedor activo (cambio manual).
SelectedModel	string	gemini-3.5-flash-lite	Modelo de Gemini.
SelectedModel_Pollinations	string	openai	Modelo de Pollinations.
profile_name, profile_program, profile_institution	string (máx. 60)	—	Perfil del estudiante (no se envía a la IA).
responsible_use_notice_accepted	bool	true	El aviso de uso responsable ya se mostró.

SecureStorage (cifrado por el sistema operativo)

Clave	Contenido
ai_api_key	Clave de Google Gemini (nombre original, conserva claves guardadas antes).
pollinations_api_key	Clave de Pollinations.

11.3 Operaciones CRUD

Operación	Dónde se realiza	Método	Evidencia
Crear consulta	Botón *Guardar* de Consultar	DatabaseService.SaveAsync (inserta si Id = 0)	Prueba P07
Consultar	Historial (búsqueda y filtro), Detalle de proyecto, Perfil (estadísticas)	GetItemsAsync, GetItemsByProjectAsync, GetStatsAsync	Pruebas P07–P09
Actualizar consulta	*Marcar como verificada* y *Guardar nota* en el detalle	SaveAsync (actualiza si Id ≠ 0)	Prueba P14
Eliminar consulta	*Eliminar* con confirmación (historial o detalle)	DeleteAsync	Prueba P08
Crear / actualizar proyecto	*Guardar proyecto*	SaveProjectAsync	Prueba P13
Eliminar proyecto	*Eliminar proyecto* (sus consultas quedan sin proyecto)	DeleteProjectAsync	Prueba P13
Eliminar	*Borrar historial* / *Borrar todos mis*	DeleteAllItemsAsync,	Prueba

Operación	Dónde se realiza	Método	Evidencia
todo	*datos locales*	DeleteAllProjectsAsync	P15

12. Gestión de API Keys

12.1 Ciclo de vida

Paso	Dónde	Qué ocurre
Crear la clave	Google AI Studio / enter.pollinations.ai	El estudiante la crea con su cuenta.
Escribirla	Configuración → campo **Clave de API** (IsPassword=True, sin corrector ni predicción)	El texto se oculta mientras se escribe.
Guardarla	ApiKeyService.SaveAsync(clave, proveedor)	Se guarda en SecureStorage con Trim(). El campo se vacía.
Consultar estado	HasKeyAsync(proveedor)	La pantalla solo dice «Clave … guardada de forma segura» o «Sin clave configurada». **La clave nunca se vuelve a mostrar.**
Usarla	AiService.AskAsync	Se lee justo antes del POST y se envía en x-goog-api-key (Gemini) o Authorization: Bearer (Pollinations).
Probarla	Botón **Probar conexión**	Envía una consulta corta y muestra el resultado o el error (sin la clave).
Eliminarla	Botón **Eliminar clave** (con confirmación)	SecureStorage.Remove del proveedor seleccionado.

12.2 Dónde se guarda en cada plataforma

Plataforma	Mecanismo de SecureStorage (.NET MAUI)
Android	EncryptedSharedPreferences con clave en **Android Keystore**. Además android:allowBackup="false" evita copias de seguridad de los datos de la app.
Windows	Cifrado con **DataProtectionProvider** asociado al usuario de Windows (datos de la app no empaquetada).
iOS / Mac Catalyst	**Llavero (Keychain)**. *(Plataformas no compiladas en este proyecto.)*

12.3 Protección de credenciales

- Ninguna clave en el código, en appsettings, en el repositorio ni en capturas.
- La clave no viaja en la URL (no queda en historiales ni registros de proxy).
- Los mensajes de error se construyen con textos fijos (AiErrorMapper), nunca con la clave.
- SensitiveDataDetector avisa si el estudiante pega una clave (AIza…, sk-…) dentro de una consulta.
- Una clave independiente por proveedor (ai_api_key, pollinations_api_key).

- RequestLimiter (8 consultas por minuto) limita el gasto de cuota si alguien usa el equipo.

El análisis de riesgos y la auditoría de las claves están en el capítulo 13.

13. Seguridad, privacidad y uso responsable de la IA

13.1 Matriz de riesgos: privacidad, seguridad y uso responsable

Escala: Probabilidad (P) y Impacto (I) de 1 (bajo) a 3 (alto). Nivel = P × I.

#	Categoría	Riesgo	P	I	Nivel	Control implementado	Dónde
R01	Seguridad	Exposición de la clave de API en el código o el repositorio	2	3	6	La clave se escribe en la app y se guarda en SecureStorage; no existe en el código ni en Git	ApiKeyService
R02	Seguridad	La clave aparece en registros o URL	2	3	6	Viaja en el encabezado x-goog-api-key (Gemini) o Authorization: Bearer (Pollinations) sobre HTTPS; los mensajes de error nunca la incluyen	AiService
R03	Seguridad	La clave queda visible en pantalla o en el teclado	2	2	4	Entry IsPassword, sin corrector ni predicción; el campo nunca se rellena con la clave guardada	SettingsPage
R04	Seguridad	Copia de seguridad en la nube de la base de datos	1	2	2	android:allowBackup="false" y tráfico en claro deshabilitado	AndroidManifest.xml
R05	Seguridad	Consumo excesivo o accidental de la cuota	2	2	4	Límite local de 8 consultas/min, máximo 2000 caracteres de entrada y 4096 tokens de salida, *timeout* de 60 s	RequestLimiter, AiService
R06	Privacidad	El estudiante envía datos personales al proveedor	3	3	9	Detector (correo, tarjeta, teléfono/cédula, credenciales, claves) con confirmación previa; aviso de privacidad en Inicio y Configuración	SensitiveDataDetector, ChatViewModel
R07	Privacidad	Datos locales permanecen tras dejar de usar la app	2	2	4	Botones *Borrar historial* y *Borrar todos mis datos locales*; *Eliminar clave*	ProfileViewModel, SettingsViewModel

#	Categoría	Riesgo	P	I	Nivel	Control implementado	Dónde
R08	Privacidad	El perfil (nombre) se envía a la IA	1	2	2	El perfil nunca forma parte de la solicitud; solo se envía el texto de la consulta y, si aplica, el nombre y descripción del proyecto	AiService
R09	Uso responsable	Se acepta una respuesta incorrecta o inventada	3	3	9	Aviso permanente en *Consultar*; respuestas «Sin verificar» hasta que el usuario las marque; nota para anotar la fuente; *system prompt* que pide declarar la incertidumbre	ChatPage, DetailViewModel
R10	Uso responsable	Plagio o entrega de trabajos generados sin declarar	2	3	6	Sección «Límites del uso de IA» y declaración de uso de IA del proyecto	SettingsPage, capítulo 13.3
R11	Uso responsable	Contenido inapropiado	1	2	2	*System prompt* académico; se informa si el proveedor bloquea la consulta	AiService
R12	Continuidad	Cambio o caída del proveedor de IA	2	2	4	Dos proveedores con cambio manual, modelo configurable, errores 402/404/5xx con mensajes claros	AiProviderCatalog, AiService, SettingsPage
R13	Integridad	Pérdida del historial por error	1	2	2	Eliminaciones siempre con confirmación; operaciones de base de datos con manejo de excepciones	ViewModels
R14	Autoría	No poder demostrar quién hizo qué	2	2	4	Commits descriptivos por integrante, declaración de uso de IA y sustentación individual	Capítulo 13.3

Riesgos residuales: el detector es una heurística (no detecta todo tipo de dato personal) y la verificación de las respuestas depende del criterio del estudiante; por eso el aviso se repite en las pantallas clave.

13.2 Auditoría de seguridad de API Keys

Alcance: almacenamiento, transmisión, visualización y uso de las claves de Gemini y Pollinations en el código del 3 de noviembre de 2026. **Método:** revisión del código fuente, del repositorio y de la configuración del proyecto.

13.2.1 Verificación de controles

#	Control	Evidencia en el código	Resultado
C1	No hay claves en el código	Búsqueda de AIza…, sk-…, pk_…, sk_… en todo el historial: solo aparecen dos claves **ficticias** dentro de las pruebas	✅ Cumple
C2	No hay claves en el repositorio ni en el historial Git	Revisión de todos los commits; sin archivos de configuración con secretos	✅ Cumple
C3	Almacenamiento cifrado	ApiKeyService → SecureStorage.Default	✅ Cumple
C4	Campo oculto	Entry IsPassword="True", sin corrector ni predicción	✅ Cumple
C5	La clave no se vuelve a mostrar	El ViewModel vacía ApiKey tras guardar; solo se informa si existe	✅ Cumple
C6	La clave no viaja en la URL	x-goog-api-key / Authorization: Bearer	✅ Cumple
C7	Solo HTTPS	Endpoints https://; Android usesCleartextTraffic="false"	✅ Cumple
C8	Errores sin la clave	AiErrorMapper usa textos fijos; prueba Messages_NeverContainTheKey	✅ Cumple
C9	Sin copias de seguridad en Android	android:allowBackup="false"	✅ Cumple
C10	Eliminación por el usuario	Botón *Eliminar clave* con confirmación	✅ Cumple
C11	Límite de uso	RequestLimiter: 8 solicitudes/min	✅ Cumple
C12	Aviso al pegar una clave en una consulta	SensitiveDataDetector detecta AIza… y sk-…	✅ Cumple
C13	Clave separada por proveedor	ai_api_key / pollinations_api_key	✅ Cumple
C14	Registros (logs) sin la clave	No se registran encabezados; AddDebug solo en DEBUG	✅ Cumple

13.2.2 Matriz de riesgos de credenciales

Riesgo	Prob.	Impacto	Nivel	Mitigación implementada	Riesgo residual
Clave publicada en GitHub	Baja	Alto	Medio	C1, C2, .gitignore	Bajo
Clave visible en capturas o video	Media	Alto	Alto	C4, C5	Bajo
Robo del archivo de la app / respaldo	Baja	Alto	Medio	C3, C9	Bajo
Interceptación en red	Baja	Alto	Medio	C6, C7	Bajo
Clave en registros o mensajes	Baja	Medio	Bajo	C8, C14	Bajo
Gasto de cuota por uso indebido del equipo	Media	Medio	Medio	C11	Medio (el equipo desbloqueado sigue pudiendo consultar)
Clave pegada en una consulta y enviada al proveedor	Media	Alto	Alto	C12 (aviso con confirmación)	Medio (es una heurística)
Ingeniería inversa del APK para extraer claves	—	—	—	No aplica: la app no incluye claves; cada usuario usa la suya	Nulo

13.2.3 Hallazgos y recomendaciones

ID	Hallazgo	Severidad	Recomendación
H1	La clave se lee en memoria como string y permanece hasta que el recolector la libera.	Baja	Aceptable en una app cliente; no guardarla en campos estáticos (ya se cumple).
H2	En Windows no empaquetado, el cifrado depende de la cuenta de usuario de Windows.	Baja	Recomendar cuentas con contraseña y no compartir el usuario.
H3	No hay rotación ni caducidad automática de claves.	Baja	Indicar en el manual cómo revocar la clave en Google AI Studio / Pollinations si se filtra.
H4	Las claves son personales y se usan desde el cliente (sin servidor intermedio).	Media (si se publica la app)	Para uso masivo, usar un backend que guarde la clave y aplique cuotas por usuario.

ID	Hallazgo	Severidad	Recomendación
H5	Las dependencias pueden tener vulnerabilidades nuevas.	Baja	Ejecutar `dotnet list package --vulnerable` antes de cada entrega.

Conclusión: no se encontraron claves expuestas y los 14 controles se cumplen. Los riesgos residuales son bajos para un proyecto académico de uso personal.

13.3 Declaración de uso de herramientas de IA

El equipo declara que usó las siguientes herramientas de IA durante el proyecto:

- **Claude (Anthropic):** apoyo para depurar errores de compilación y de entorno, generar y revisar código C# y XAML (incluida la integración del proveedor Pollinations, el manejo de errores 402/408 y sus pruebas), proponer el diseño visual y redactar borradores de esta documentación.
- **Google Gemini y Pollinations:** servicios de IA que consume la aplicación en su funcionamiento; no se usaron para escribir el código.

El equipo revisó el código y la documentación producidos con ayuda de IA, y cada integrante puede explicar el recorrido completo de una consulta (capítulo 5.5) y justificar las decisiones de seguridad.

Fuentes técnicas consultadas: Microsoft Learn (.NET MAUI, `HttpClient`, inyección de dependencias, `SecureStorage`), documentación de CommunityToolkit.Mvvm, sqlite-net, Google Gemini API y Pollinations API.

14. Pruebas y calidad

14.1 Plan de pruebas manuales

Dispositivo: Windows (Windows Machine) · **Responsables:** Juan Carlos Díaz Cifuentes y Sebastián Grande Moreno.

Caso	Condición y pasos	Resultado esperado	Resultado obtenido	Estado
P01	**Prompt válido y clave válida.** Configurar clave → Consultar «Explica qué es una API REST»	Respuesta visible y estado final correcto	Se consultó «Explica qué es una API REST»; la respuesta se muestra y aparece «Respuesta recibida…».	Cumple (evidencia a)
P02	**Prompt vacío.** Dejar el editor vacío → Consultar	No se envía la solicitud	Mensaje «Escriba una consulta antes de enviar.»; no se realiza la petición.	Cumple (evidencia b)
P03	**Sin conexión.** Desconectar la red → Consultar	La aplicación no se cierra	Mensaje «Sin conexión a internet. Verifique su red e intente de nuevo.»	Cumple (evidencia c)
P04	**Clave inválida.** Guardar una clave falsa → Probar conexión	Error de autorización sin mostrar la clave	«Error de autorización (400): la clave de API no es válida.» La clave no se muestra.	Cumple (evidencia d)
P05	**Tiempo agotado.** Reducir temporalmente el tiempo máximo del HttpClient → Consultar	La espera termina y se puede reintentar	Aparece «Tiempo de espera agotado. Intente de nuevo.»; los botones siguen	Cumple (evidencia e)

Caso	Condición y pasos	Resultado esperado	Resultado obtenido	Estado
			activos. Luego se restauró a 60 s.	
P06	**Cancelar solicitud.** Consultar y pulsar Cancelar	Se cancela sin bloquear la interfaz	Mensaje «Consulta cancelada.»; los botones siguen activos.	Cumple (evidencia f)
P07	**Guardar conversación.** Obtener respuesta → Guardar → Historial	El registro aparece en el historial	La consulta guardada aparece al inicio de la lista del historial.	Cumple (evidencia g)
P08	**Eliminar conversación.** Historial → Eliminar → confirmar	Se solicita confirmación y se actualiza la lista	La lista pasa de 3 a 2 elementos al eliminar la consulta sobre motos.	Cumple (evidencia h)
P09	**Reiniciar la aplicación.** Guardar, cerrar por completo y abrir	El historial permanece	Las consultas guardadas siguen en el historial y aparece también la consulta nueva.	Cumple (evidencia i)
P10	**Pantalla pequeña / ventana estrecha.** Reducir la ventana en Windows o usar un teléfono pequeño	Todos los controles son accesibles (desplazamiento)		Por ejecutar
P11	**Datos sensibles.** Escribir «mi correo es a@b.com» → Consultar	Aparece el aviso; *Revisar* cancela el envío		Por ejecutar
P12	**Límite de uso.** Enviar 9 consultas en menos de un minuto	La novena muestra «Límite de uso alcanzado…»		Por ejecutar
P13	**Proyectos.** Crear, editar y eliminar un proyecto	Los cambios se reflejan; sus consultas siguen en el historial		Por ejecutar
P14	**Nota personal.** Editar la nota de	La nota persiste		Por

Caso	Condición y pasos	Resultado esperado	Resultado obtenido	Estado
	una consulta y reabrirla			ejecutar
P15	**Borrar todos los datos.** Perfil → Borrar todos mis datos locales	Historial, proyectos y perfil quedan vacíos		Por ejecutar
P16	**Cambiar a Pollinations.** Configuración → Pollinations → clave → Guardar → Probar conexión	«✔️ Conexión exitosa con Pollinations»		Por ejecutar
P17	**Consultar con Pollinations.** Consultar con Pollinations activo	Respuesta visible y «Respuesta recibida de Pollinations»		Por ejecutar
P18	**Volver a Gemini.** Seleccionar Gemini → Guardar (sin escribir la clave) → Consultar	Funciona: cada proveedor conserva su clave		Por ejecutar
P19	**Cambio sin guardar.** Cambiar el selector sin pulsar Guardar → Consultar	Se sigue usando el proveedor activo		Por ejecutar
P20	**Clave inválida en Pollinations.** Guardar una clave falsa de Pollinations → Probar conexión	«Error de autorización (401) …» sin mostrar la clave		Por ejecutar

14.2 Evidencias de las pruebas manuales

a) Consulta informativa exitosa (P01)

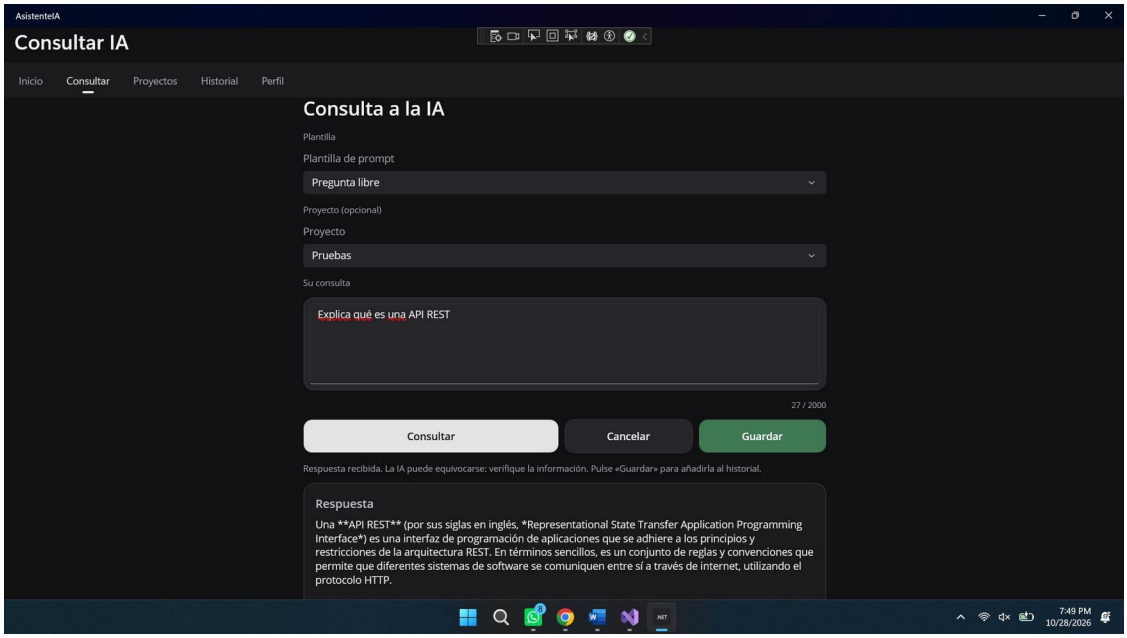


Figura 13. Consulta informativa exitosa (P01).

b) Error si no se escribe nada (P02)

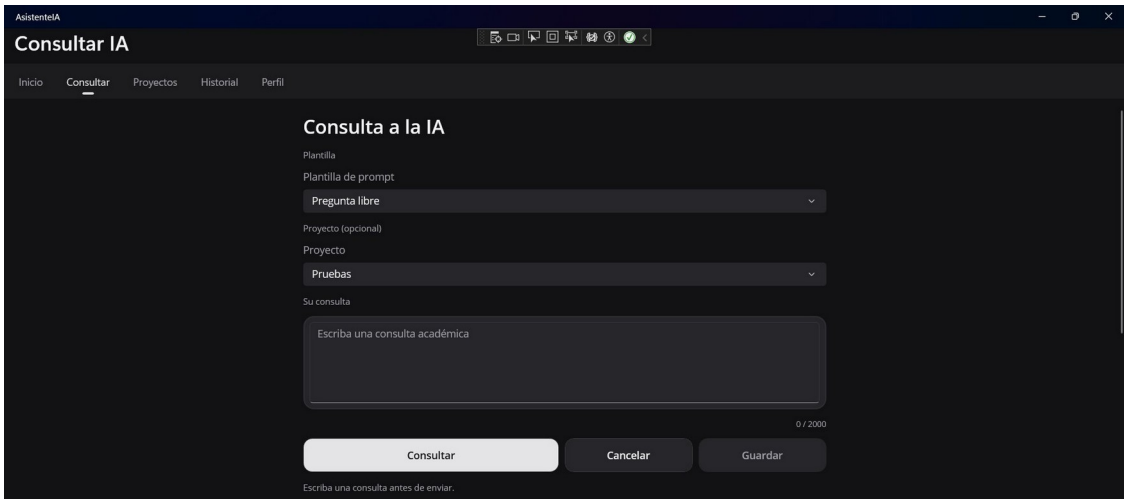


Figura 14. Validación de consulta vacía (P02).

c) Error sin internet (P03)

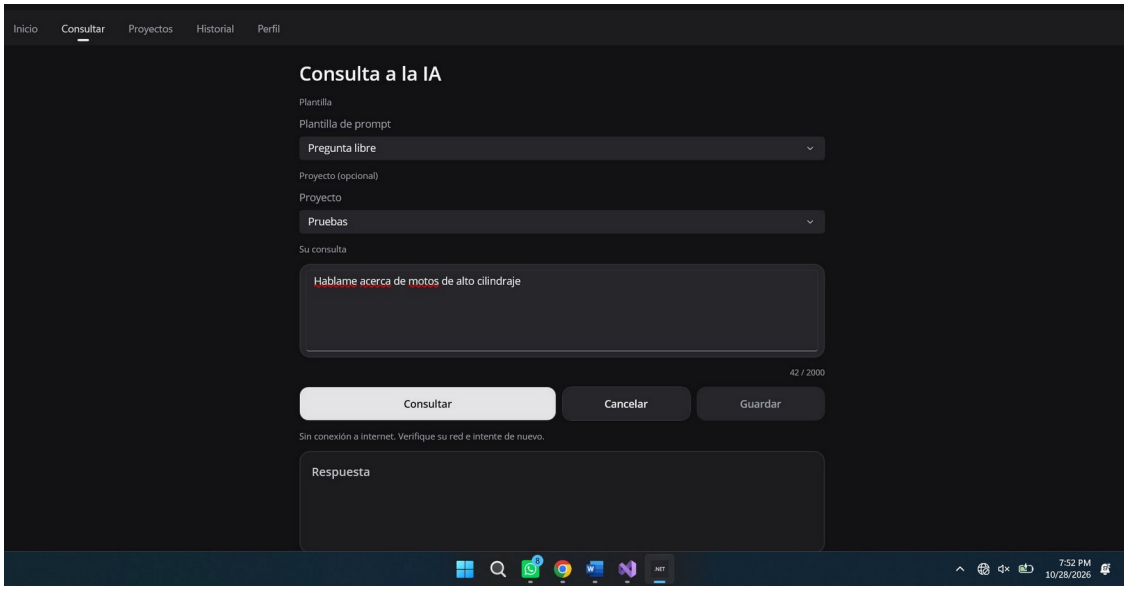


Figura 15. Prueba de error sin conexión a internet (P03).

d) Error con clave de API falsa (P04)

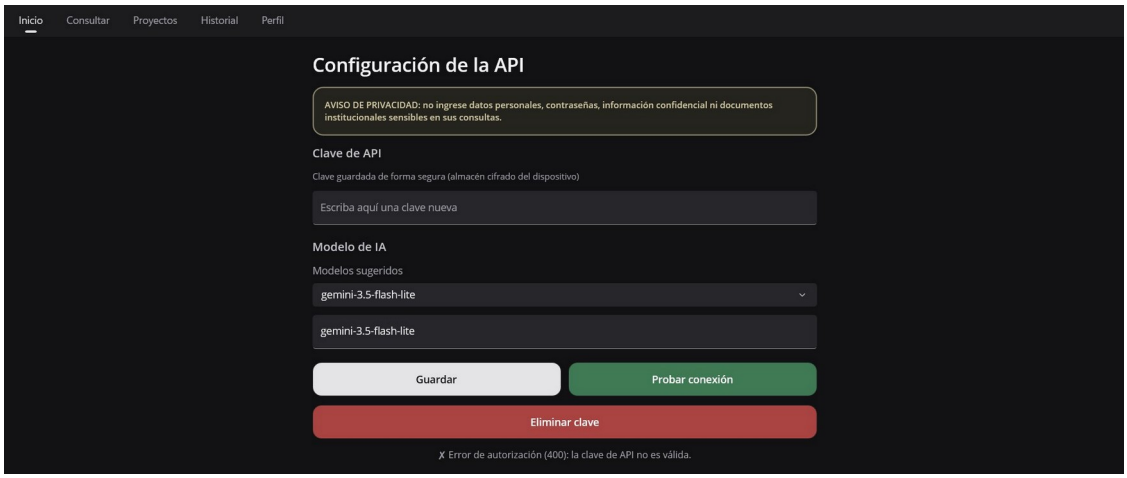


Figura 16. Prueba de error con clave de API inválida (P04).

e) Tiempo de espera agotado (P05)

Para esta prueba se redujo temporalmente el tiempo máximo de espera del HttpClient y luego se restauró a 60 segundos.

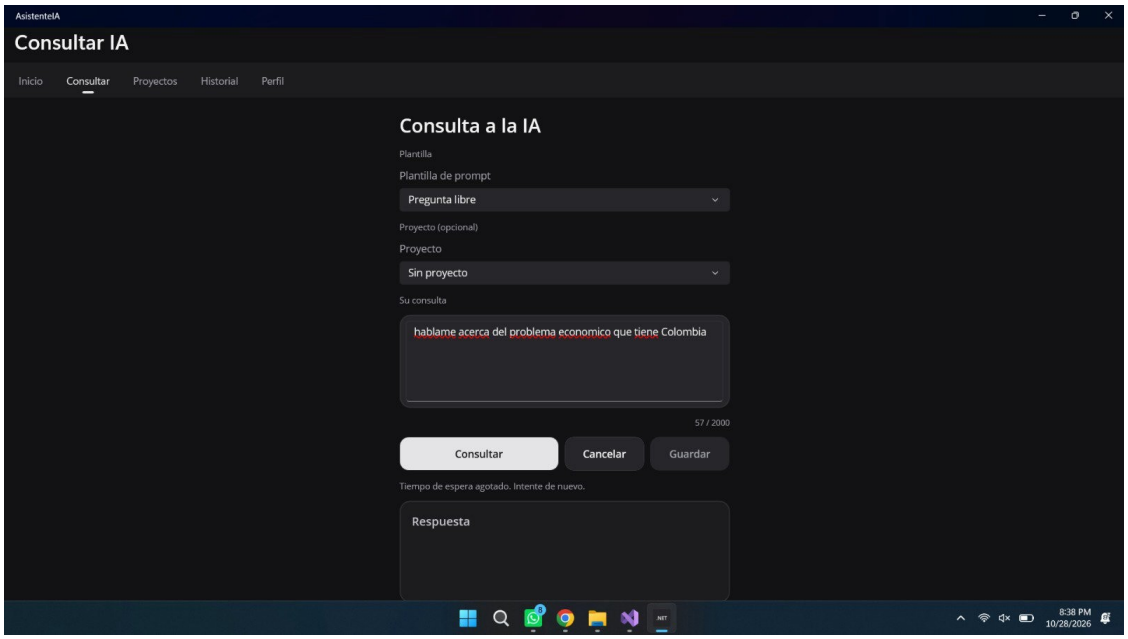


Figura 17. Prueba de tiempo de espera agotado (P05).

f) Consulta cancelada (P06)

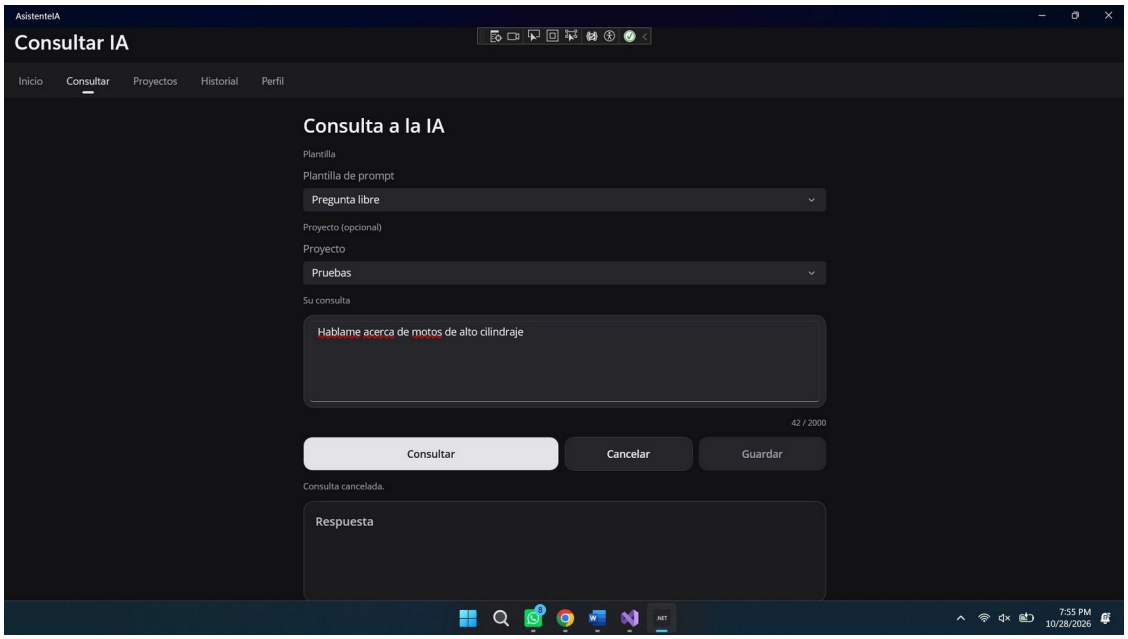


Figura 18. Prueba de consulta cancelada (P06).

g) Historial de consultas (P07)

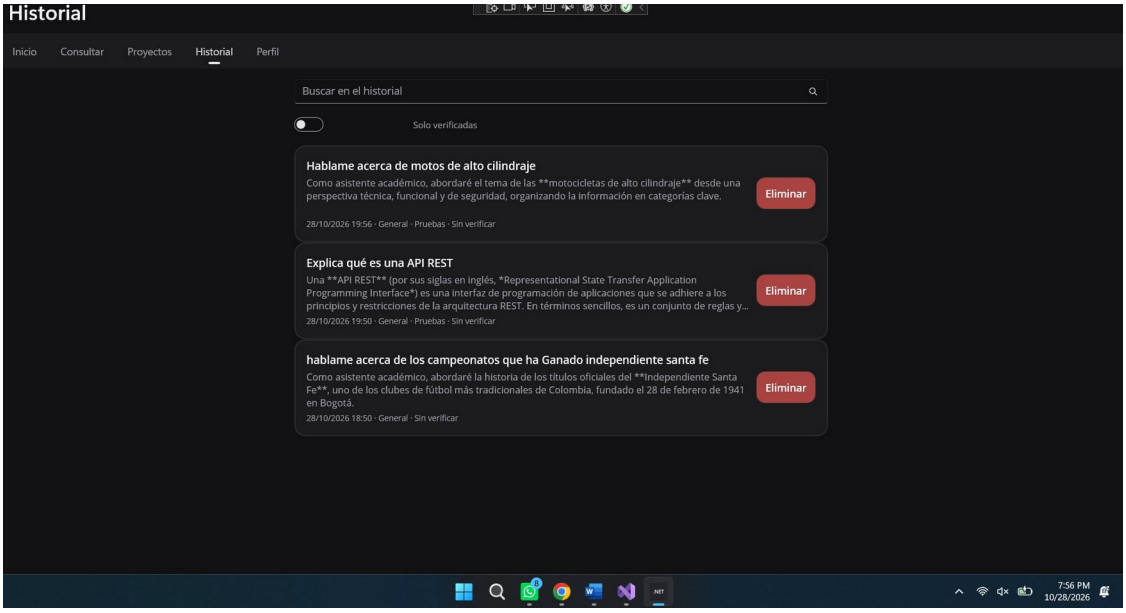


Figura 19. Historial con tres consultas guardadas (P07).

h) Eliminación de una consulta del historial (P08)

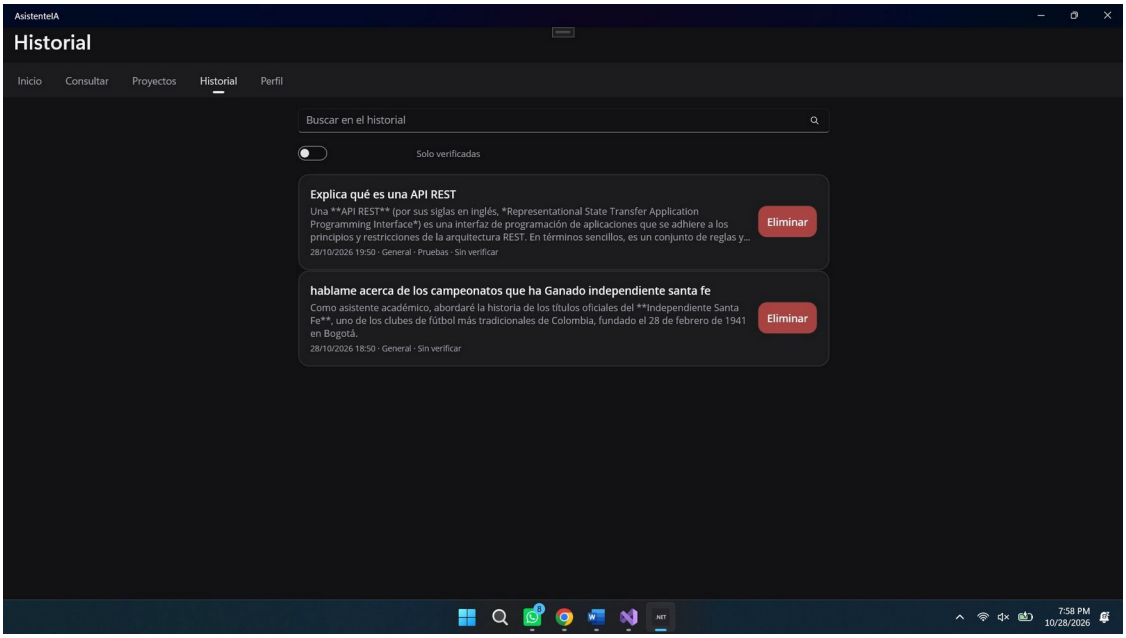


Figura 20. Historial después de eliminar una consulta (P08).

i) Persistencia al reiniciar la aplicación (P09)

El historial antes de cerrar la aplicación y después de volver a abrirla: las consultas anteriores se conservan y aparece la consulta nueva.

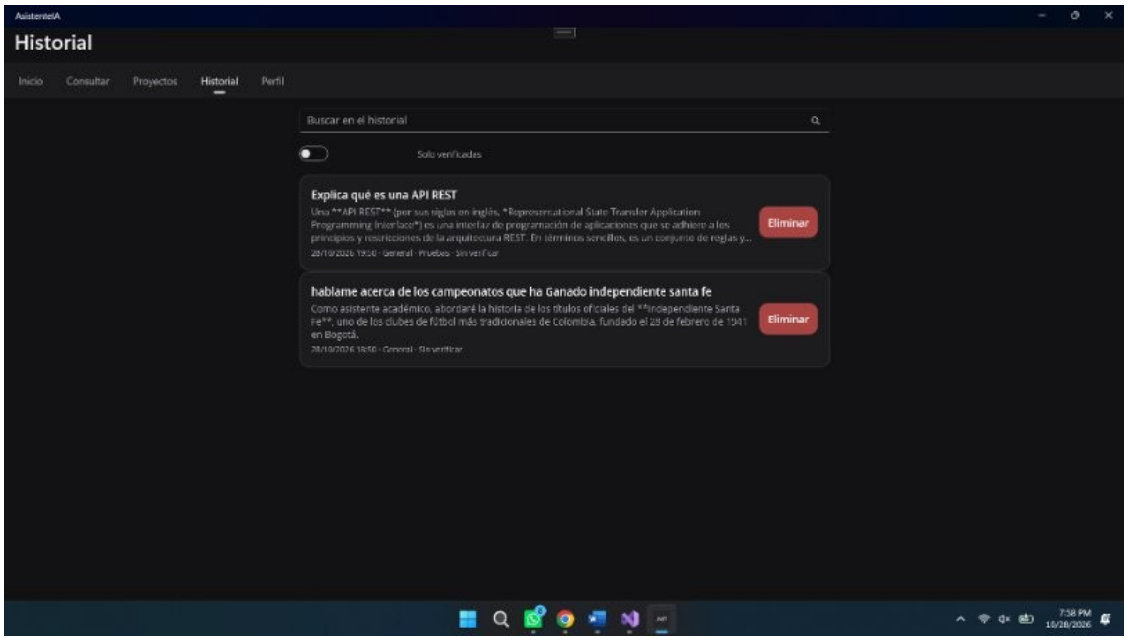


Figura 21. Historial antes de cerrar la aplicación (P09).

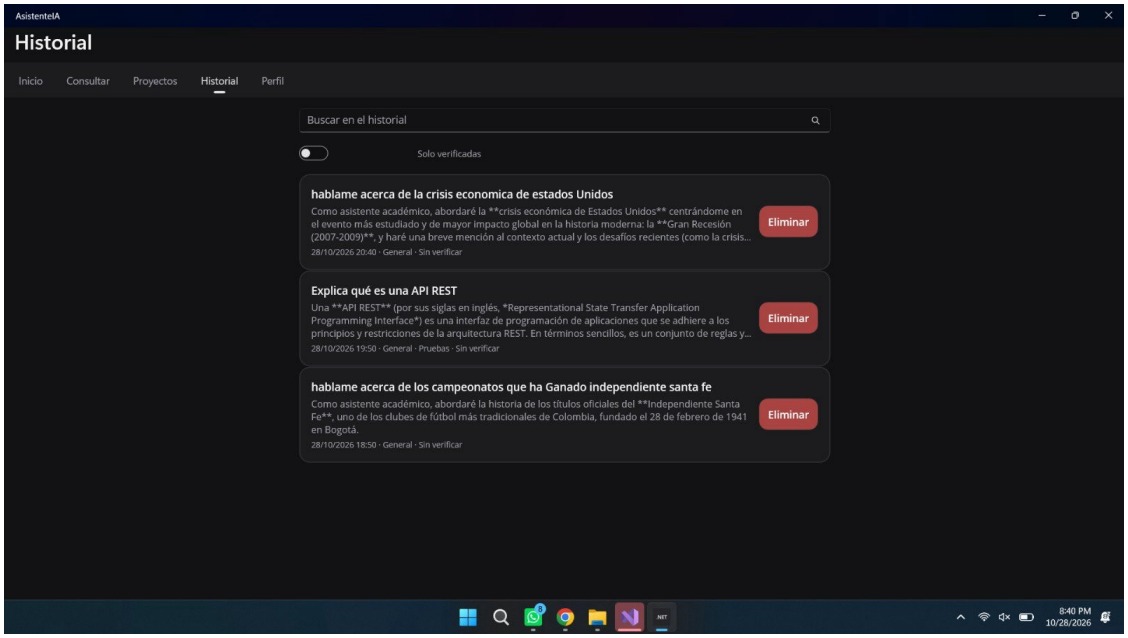


Figura 22. Historial después de cerrar y volver a abrir la aplicación (P09).

14.3 Pruebas automáticas

El proyecto **AsistenteIA.Tests** (xUnit, net9.0) prueba la lógica que no depende de MAUI. Se ejecuta con *Prueba › Explorador de pruebas › Ejecutar todas* o con `dotnet test AsistenteIA.Tests`.

Clase	Casos	Qué prueba
PromptValidatorTes	8	Vacío, corto, mínimo, máximo y sobre el máximo (apoya P02).

Clase	Casos	Qué prueba
ts		
SensitiveDataDetectorTests	18	Correo, tarjeta (Luhn), teléfono, cédula, credenciales, clave de API y falsos positivos (apoya P11).
RequestLimiterTests	3	Permite hasta el límite, bloquea e informa la espera, se libera tras la ventana (apoya P12).
UserProfileTests	5	Iniciales del avatar.
AiProviderCatalogTests	16	Catálogo de proveedores, lectura de la preferencia, modelos, endpoints, clave en encabezado (apoya P16–P19).
AiErrorMapperTests	16	Mensajes para 400, 401, 402, 403, 404, 408, 429, 5xx, sin conexión, tiempo agotado, cancelación y que nunca contienen la clave (apoya P03–P06, P20).
Total	**66**	

La Figura siguiente muestra la ejecución en el Explorador de pruebas de Visual Studio de las 34 pruebas de las cuatro primeras clases: 34 correctas, 0 fallidas, 0 omitidas, duración total 0,5 s. Las 32 pruebas de AiProviderCatalogTests y AiErrorMapperTests se agregaron con la integración de Pollinations y se ejecutan con el mismo comando.

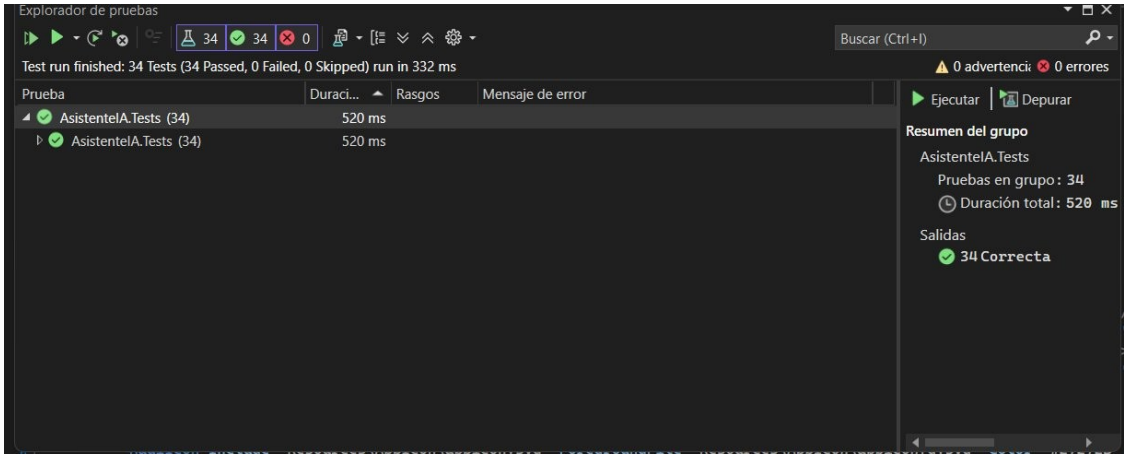


Figura 23. Resultado de las pruebas automáticas en el Explorador de pruebas.

Las pruebas automáticas complementan a las manuales: verifican la lógica de forma repetible, sin depender de la interfaz ni de la conexión.

15. Manual de usuario

AsistenteIA es un asistente académico: envía tus preguntas a un modelo de inteligencia artificial, muestra la respuesta y guarda un historial **solo en tu dispositivo**. Funciona en Windows y Android.

15.1 Primer uso

1. Abre la app. La primera vez aparece el aviso **«Uso responsable de la IA»**; léelo y pulsa **Entendido**.
2. En **Inicio** pulsa **Configurar clave y modelo** (o ve a **Perfil › Configuración**).
3. Elige el **proveedor**: *Google Gemini* o *Pollinations*.
4. Escribe tu **clave de API** (se oculta) y elige un **modelo** sugerido.
5. Pulsa **Guardar** y luego **Probar conexión**. Debe aparecer «✔️ Conexión exitosa con {proveedor}».

Clave de Gemini: Google AI Studio (aistudio.google.com/apikey). Clave de Pollinations: enter.pollinations.ai.

15.2 Pantallas (ver capturas en el capítulo 8)

Pantalla	Para qué sirve	Acciones
Inicio	Bienvenida con tu nombre, accesos rápidos y recomendaciones.	Nueva consulta, Mis proyectos, Historial, Mi perfil, Configurar clave y modelo.
Consultar	Hacer preguntas a la IA.	Elegir plantilla y proyecto, escribir (contador hasta 2000), **Consultar**, **Cancelar**, **Guardar**.
Proyectos	Agrupar consultas por materia o trabajo.	**+ Nuevo proyecto**, abrir un proyecto.
Detalle de proyecto	Ver y editar un proyecto.	Editar nombre (3–60) y descripción (máx. 300), ver sus consultas, **Consultar en este proyecto**, eliminar.
Historial	Ver consultas guardadas.	Buscar (en pregunta, respuesta, categoría o proyecto), **Solo verificadas**, abrir detalle, **Eliminar**.
Detalle de consulta	Revisar una consulta.	**Marcar como verificada / Quitar verificación**, escribir una **nota personal** (máx. 500) y pulsar **Guardar nota**, **Eliminar consulta**.
Perfil	Tus datos y estadísticas.	Nombre, programa e institución; contadores de consultas, verificadas y proyectos; Configuración; **Borrar historial de consultas**; **Borrar todos mis datos locales**.

Pantalla	Para qué sirve	Acciones
Configuración	Proveedor, clave y modelo.	Selector de proveedor, clave oculta, modelos, Guardar, Probar conexión, Eliminar clave; avisos de privacidad y «Acerca de».

15.3 Hacer una consulta

1. Ve a **Consultar**.
2. (Opcional) Elige una **plantilla**: Pregunta libre, Explicar un concepto, Resumir un texto, Comparar dos temas, Preguntas de práctica o Revisar código. El texto aparece en el editor y puedes cambiarlo.
3. (Opcional) Elige un **proyecto**: su nombre y descripción se envían como contexto.
4. Escribe tu pregunta (5 a 2000 caracteres) y pulsa **Consultar**. Verás «Consultando {proveedor}…».
5. Si tarda, pulsa **Cancelar**. La espera máxima es 60 segundos.
6. Lee la respuesta, **verifícala** con fuentes confiables y pulsa **Guardar** para añadirla al historial.

Avisos que puedes ver

Aviso	Qué significa
«Posibles datos sensibles…»	Tu texto parece tener un correo, teléfono, cédula, tarjeta, contraseña o clave. Pulsa **Revisar** para quitarlos o **Enviar** si estás seguro.
«Límite de uso alcanzado: espere N s»	Hiciste más de 8 consultas en un minuto.
«[Respuesta recortada…]»	La respuesta superó el límite de longitud: pide un resumen o divide la pregunta.

15.4 Cambiar de proveedor

En **Configuración** elige el otro proveedor, escribe su clave (solo la primera vez), elige el modelo y pulsa **Guardar**. Cada proveedor guarda su propia clave y su propio modelo. La línea «Proveedor activo» muestra cuál se está usando.

15.5 Privacidad y datos

- Tu clave se guarda cifrada en el dispositivo y nunca se muestra.
- El historial, los proyectos y el perfil se guardan solo en tu dispositivo. El perfil **no** se envía a la IA.
- Lo único que se envía al proveedor es el texto de la consulta (y el contexto del proyecto, si elegiste uno).
- **Borrar historial de consultas** elimina las consultas; **Borrar todos mis datos locales** elimina consultas, proyectos y perfil. La clave se elimina en Configuración.

15.6 Recomendaciones

- No escribas datos personales, contraseñas ni información confidencial.
- La IA puede equivocarse o inventar datos: márcala como verificada solo después de contrastarla.
- Si ves un error, consulta el Manual de solución de problemas (capítulo 17).

16. Manual técnico para desarrolladores

16.1 Organización del código

```
AsistenteIA.slnx
├── AsistenteIA/                # App .NET MAUI (Windows, Android)
│   ├── MauiProgram.cs          # Inyección de dependencias y HttpClient (60 s)
│   ├── AppShell.xaml(.cs)      # TabBar de 5 pestañas + rutas Detail, ProjectDetail, Settings
│   ├── Models/                 # ChatModels (DTO Gemini y OpenAI), ConversationRecord, Project,
│   │                           # PromptTemplate, UserProfile, AiProvider (catálogo de proveedores)
│   ├── Services/               # AiService, AiErrorMapper, ApiKeyService, DatabaseService, ProfileService,
│   │                           # PromptValidator, SensitiveDataDetector, RequestLimiter,
│   │                           # INavigationService, IDialogService
│   ├── ViewModels/             # 8 ViewModels con CommunityToolkit.Mvvm
│   ├── Views/                  # 8 páginas XAML (solo binding)
│   ├── Resources/              # Colors.xaml, Styles.xaml, fuentes, íconos
│   └── Platforms/              # Android (manifest), Windows, iOS, MacCatalyst
├── AsistenteIA.Tests/          # xUnit (net9.0) — enlaza los archivos de lógica pura
└── docs/                       # Documentación (01–07 y documentacion-final/)
```

16.2 Patrones y reglas

Regla	Detalle
MVVM	Las vistas no tienen lógica; los ViewModels no usan `Shell` directamente (usan `INavigationService` e `IDialogService`).
Inyección	Servicios *singleton*; ViewModels y páginas *transient*; las páginas reciben su ViewModel por constructor.
Asincronía	Toda E/S es `async`; las consultas aceptan `CancellationToken`.
Lógica probable	Lo que no depende de MAUI va en clases puras y se enlaza en `AsistenteIA.Tests.csproj` con `<Compile Include … Link=…>`.
Configuración	Proveedor y modelo en `Preferences`; claves en `SecureStorage`; nada en el código.

16.3 Inyección de dependencias (`MauiProgram.cs`)

Registro	Ciclo de vida	Motivo
`HttpClient` (BaseAddress de Gemini, Timeout 60 s)	Singleton	Reutiliza conexiones.
`RequestLimiter`, `ApiKeyService`, `AiService`, `DatabaseService`, `ProfileService`	Singleton	Estado compartido (límite, conexión SQLite).
`INavigationService` → `ShellNavigationService`, `IDialogService` → `ShellDialogService`	Singleton	Abstraen Shell para los ViewModels.
8 ViewModels y 8 páginas	Transient	Cada pantalla empieza limpia.

16.4 Configuración y ejecución

```
git clone https://github.com/sebitazz321/ASISTENTEIA.git
cd ASISTENTEIA
dotnet workload install maui          # si no usa Visual Studio
dotnet build AsistenteIA/AsistenteIA.csproj -f net9.0-windows10.0.19041.0
dotnet build AsistenteIA/AsistenteIA.csproj -f net9.0-android
```

En Visual Studio: abrir AsistenteIA.slnx, elegir **Windows Machine** (o un emulador Android) y pulsar **Ejecutar**. El ejecutable de depuración queda en AsistenteIA/bin/Debug/net9.0-windows10.0.19041.0/win10-x64/AsistenteIA.exe (la arquitectura puede variar).

16.5 Pruebas

```
dotnet test AsistenteIA.Tests      # 66 pruebas: validador, datos sensibles, límite, perfil, proveedores, errores
```

Clase	Casos	Cubre
PromptValidatorTests	8	RF02
SensitiveDataDetectorTests	18	Privacidad
RequestLimiterTests	3	Límite de uso
UserProfileTests	5	Iniciales del avatar
AiProviderCatalogTests	16	Catálogo, parseo, modelos, endpoints, claves en encabezado
AiErrorMapperTests	16	400, 401, 402, 403, 404, 408, 429, 5xx, conexión, cancelación, sin clave en mensajes

Las pruebas manuales P01–P20 están en el capítulo 14.

16.6 Mantenimiento

Tarea	Cómo
Cambiar los modelos sugeridos	AiProviderCatalog (SuggestedModels, DefaultModel) y su prueba DefaultModel_IsAmongSuggested.
Agregar un proveedor	Ver el capítulo 7.4.
Agregar un campo a SQLite	Propiedad en la entidad; CreateTableAsync agrega la columna a bases existentes (no borra datos).
Cambiar el límite de uso	RequestLimiter.DefaultMaxRequests / DefaultWindow.
Cambiar el tiempo máximo	Timeout del HttpClient en MauiProgram.cs.
Actualizar paquetes	dotnet list package --outdated y --vulnerable; probar y registrar en el historial de cambios.
Migrar a .NET 10	TFM net10.0-android;net10.0-windows10.0.19041.0, global.json 10.0.100,

Tarea	Cómo
	actualizar paquetes y ejecutar las pruebas.
Entregar sin binarios	git archive --format=zip --output=../AsistenteIA-entrega.zip HEAD

17. Manual de solución de problemas

17.1 Instalación

Problema	Causa probable	Solución
Visual Studio no muestra proyectos MAUI	Falta la carga de trabajo	Visual Studio Installer → Modificar → «Desarrollo de la interfaz de usuario de aplicaciones multiplataforma de .NET».
NETSDK1045 / «el SDK actual no admite .NET 9»	SDK antiguo	Instalar .NET SDK 9.0.1xx o superior; dotnet --list-sdks para verificar.
A compatible .NET SDK was not found (global.json)	No hay SDK 9.0	Instalar el SDK 9 o, si migró a .NET 10, actualizar global.json.
workload maui not installed	Falta el workload por consola	dotnet workload install maui (como administrador).
No aparecen emuladores Android	SDK/emulador no instalado	Herramientas → Android → Administrador de dispositivos → crear un dispositivo API 34.

17.2 Compilación

Problema	Causa probable	Solución
Unable to load DLL 'e_sqlite3' en Windows	Faltan componentes C++	Instalar «Herramientas de compilación de C++ para escritorio» y recompilar.
Errores de paquetes NuGet tras cambiar de equipo	Caché o restauración incompleta	Clic derecho en la solución → Restaurar paquetes NuGet; o dotnet restore.
Errores raros después de cambiar de rama	Carpetas bin/obj viejas	Cerrar VS, borrar bin y obj de ambos proyectos, compilar de nuevo.
XFC0000 / errores XAML	Binding o recurso mal escrito	Revisar la línea indicada; los estilos están en Resources/Styles.
Las pruebas no aparecen en el Explorador	Falta compilar el proyecto de pruebas	Compilar AsistenteIA.Tests y pulsar «Ejecutar todas»; o dotnet test AsistenteIA.Tests.
Archivo de la app en uso al compilar	La app sigue abierta	Cerrar la app o detener la depuración.

17.3 Conexión con la IA

Mensaje	Causa	Solución
«Sin conexión a	Sin red	Revisar Wi-Fi/datos y volver a intentar.

Mensaje	Causa	Solución
internet…»		
«No se pudo conectar con el servicio…»	DNS, proxy o firewall bloquean el dominio	Permitir generativelanguage.googleapis.com y gen.pollinations.ai (puerto 443).
«Tiempo de espera agotado…»	Respuesta muy lenta (> 60 s)	Reintentar, acortar la consulta o cambiar de modelo/proveedor.
«Error de autorización (400/401/403)…»	Clave inválida, revocada o de otro proveedor	Configuración → elegir el proveedor correcto → pegar una clave nueva → Guardar → Probar conexión.
«…saldo o límite gratuito (402)»	Pollinations sin saldo	Revisar la cuenta en enter.pollinations.ai o cambiar a Gemini.
«Modelo no encontrado (404)…»	Modelo retirado o mal escrito	Elegir un modelo de la lista sugerida.
«Se superó la cuota… (429)»	Límite del proveedor	Esperar unos minutos.
«Límite de uso alcanzado: espere N s»	Más de 8 consultas por minuto (límite de la app)	Esperar el tiempo indicado.
«El servicio de IA no está disponible (5xx)»	Falla del proveedor	Reintentar más tarde o cambiar de proveedor.
«El servicio bloqueó la consulta (…)»	Filtro de seguridad de Gemini	Reformular la pregunta.

17.4 Configuración y datos

Problema	Causa	Solución
«Configure la clave de {proveedor}…»	El proveedor activo no tiene clave	Configuración → escribir la clave de ese proveedor → Guardar.
Cambié de proveedor y sigue usando el anterior	No se pulsó Guardar	El selector solo prepara el cambio; pulse **Guardar** y revise «Proveedor activo».
«No se pudo leer el almacén seguro»	SecureStorage dañado (p. ej. tras restaurar el sistema)	Eliminar la clave y guardarla de nuevo; en Android, borrar datos de la app.
El historial desapareció	Se usó «Borrar todos mis datos locales» o se reinstaló la app	Los datos son locales; no hay copia en la nube.
El aviso de uso responsable no vuelve a salir	Ya se aceptó	Es normal; se muestra una sola vez.

18. Historial de cambios del proyecto

Construido a partir del historial Git (git log --stat).

Versión	Fecha	Commit	Autor registrado en Git	Cambios	Archivos afectados
0.1	22/10/2026	4fc6329	Juan Rodriguez	Agrega .gitattributes y .gitignore.	2 archivos
0.2	22/10/2026	5844a08	Juan Rodriguez	Agrega los archivos del proyecto (plantilla .NET MAUI).	38 archivos
0.3	22/10/2026	20b0bb9	Juan Rodriguez	Base inicial: modelos, servicios, ViewModels y vistas.	56 archivos
1.0	03/11/2026	17ab990	Sebastian Grande Moreno	Versión mejorada: Proyectos, Detalle de proyecto, Perfil, navegación y diálogos inyectables, detector de datos sensibles, límite de uso, notas, 34 pruebas xUnit y documentación docs/01–07. Une las ramas 20b0bb9 y 5844a08.	63 archivos (34 nuevos, 2 eliminados, 27 modificados)
1.1	03/11/2026	fb45b32	Sebastian Grande Moreno	Proveedor Pollinations y cambio manual de proveedor; clave y modelo por proveedor; AiErrorMapper con 402 y 408; selector en Configuración; autoría en «Acerca de».	AiProvider.cs, ChatModels.cs, AiErrorMapper.cs, AiService.cs, ApiKeyService.cs, ChatViewModel.cs, SettingsViewModel.cs, SettingsPage.xaml
1.2	03/11/2026	76bd256	Sebastian Grande Moreno	32 pruebas de proveedores y errores (total 66).	AiProviderCatalogTests.cs, AiErrorMapperTests.cs, AsistenteIA.Tests.csproj
1.3	03/11/2026	ad5cbe7y	Sebastian Grande	Documentación final: diagramas, diccionario de	docs/documentacion-final/**,

Versión	Fecha	Commit	Autor registrado en Git	Cambios	Archivos afectados
		siguientes	Moreno	datos, API REST, proveedores, API Keys, errores, auditoría, manuales, historial, Git y conclusiones; documento consolidado en Word y PDF.	docs/README.md

Responsables por área

Integrante	Áreas
Juan Carlos Díaz Cifuentes	API REST, almacenamiento seguro de la clave, SQLite, pruebas y documentación.
Sebastián Grande Moreno	Navegación, pantallas XAML, estilos, integración de la solución, proveedores de IA y repositorio.

19. Trabajo en equipo y repositorio Git

19.1 Roles

Integrante	Rol principal	Aportes
Juan Carlos Díaz Cifuentes	Servicios y datos; calidad y documentación	API REST, almacenamiento seguro de la clave, SQLite, pruebas y documentación.
Sebastián Grande Moreno	Líder e integración; interfaz y accesibilidad	Navegación, pantallas XAML, estilos, integración de la solución, proveedores de IA y repositorio.

Los roles siguen la organización propuesta por la guía: líder e integración; interfaz y accesibilidad; servicios y datos; calidad y documentación. Ambos integrantes revisaron el código y comprenden el recorrido completo de la aplicación.

19.2 Repositorio

Elemento	Valor
URL	https://github.com/sebitazz321/ASISTENTEIA
Rama principal	main
Exclusiones (.gitignore)	bin/, obj/, .vs/, *.user

Ruta	Contenido
AsistenteIA/	Aplicación .NET MAUI
AsistenteIA.Tests/	Pruebas xUnit
docs/01…07	Documentación de trabajo del equipo (Markdown)
docs/documentacion-final/	Este documento: capítulos en Markdown, diagramas (PNG y Mermaid), capturas y versión consolidada en Word y PDF

19.3 Historial de commits

```
*   (documentación final)
*   ad5cbe7   Documentación complementaria: 18 entregables
*   76bd256   Agrega 32 pruebas de proveedores y manejo de errores
*   fb45b32   Agrega proveedor Pollinations y cambio manual de proveedor de IA
*   17ab990   Integra la versión mejorada del proyecto (merge de 20b0bb9 y 5844a08)
|\
| * 5844a08   Agregar archivos de proyecto.
* | 20b0bb9   Trabajo completo Agregar archivos de proyecto.
|/
*   4fc6329   Agregar .gitattributes y .gitignore.
```

Los tres primeros commits (22/10/2026) corresponden a la base inicial del proyecto y conservan el autor que registró Git en ese momento. Los commits siguientes (03/11/2026) contienen la versión mejorada, la integración de Pollinations, las pruebas y la documentación.

19.4 Flujo de trabajo

1. Cada integrante configura su identidad antes de trabajar: git config user.name "Nombre Apellido" y git config user.email "correo".
2. Trabajo en ramas cortas (feature/..., fix/..., docs/...) con *Pull Request* hacia main.
3. Un commit por cambio lógico, en imperativo y explicando el porqué.
4. Comandos para revisar la evidencia:

```
git log --graph --oneline --all        # gráfico de ramas y commits
git shortlog -sne                      # commits por autor
git log --stat --since="2026-10-01"    # archivos afectados por commit
```

20. Lista de cotejo de entrega

Criterio	Cumple	Observación
Compila y ejecuta sin errores	Sí	Ejecución en Windows mostrada en los capítulos 8 y 14. Tras la integración de Pollinations, recompilar y ejecutar las 66 pruebas antes de la entrega.
Incluye mínimo cinco pantallas	Sí	Ocho: Inicio, Consultar, Proyectos, Detalle de proyecto, Historial, Detalle de consulta, Perfil y Configuración.
Usa Shell, XAML adaptable y MVVM	Sí	Capítulos 5 y 8.
Consume la API REST mediante POST	Sí	Gemini y Pollinations (capítulos 7 y 9).
No contiene claves expuestas	Sí	SecureStorage; auditoría con 14 controles (capítulo 13.2).
Maneja carga, cancelación y errores	Sí	Capítulo 10; pruebas P01–P06.
Guarda historial en SQLite	Sí	Capítulo 11.
Casos de prueba documentados	Sí	P01–P09 ejecutados con evidencia; P10–P20 definidos para ejecutar (capítulo 14).
Pruebas automáticas	Sí	66 pruebas xUnit (capítulo 14.3).
Diagramas de arquitectura, clases y ERD	Sí	Capítulos 5, 6 y 11.
Diccionario de datos	Sí	Capítulo 11.2.
Manuales de usuario, técnico y de solución de problemas	Sí	Capítulos 15, 16 y 17.
Declaración de uso de IA	Sí	Capítulo 13.3.
Repositorio con historial	Sí	Capítulo 19.
Todos sustentan su aporte	Sí	Sustentación individual del código por cada integrante.

21. Conclusiones y trabajo futuro

21.1 Resultados obtenidos

- Aplicación .NET MAUI funcional en Windows (y compilable para Android) con **8 pantallas**, arquitectura **MVVM** e inyección de dependencias.
- Consumo de API REST de IA con **POST + JSON**, cancelación, tiempo máximo de 60 s y manejo de errores por código HTTP.
- **Dos proveedores** (Gemini y Pollinations) con cambio manual, sin tocar vistas ni ViewModels de consulta.
- Persistencia **SQLite** con CRUD de consultas y proyectos; perfil y preferencias locales.
- Seguridad de credenciales: SecureStorage, clave en encabezado, sin claves en el repositorio (auditoría con 14 controles cumplidos).
- Uso responsable: aviso inicial, detector de datos sensibles, límite de uso, verificación manual y borrado de datos.
- **66 pruebas automáticas** sobre la lógica pura y 20 casos manuales definidos (P01–P09 ejecutados con evidencia).

21.2 Cumplimiento de objetivos

Objetivo	Resultado
Asistente académico con IA vía API REST	✅
Historial persistente en SQLite	✅
Seguridad de la clave	✅
Mínimo cinco pantallas con Shell y MVVM	✅ (8)
Manejo de carga, cancelación y errores	✅
Pruebas	✅ automáticas · ✅ P01–P09 con evidencia · P10–P20 definidos
Repositorio Git con historial	✅

21.3 Limitaciones

1. Las pruebas manuales P10–P20 y la ejecución de las 66 pruebas automáticas deben registrarse con capturas.
2. .NET 9 deja de tener soporte el 10/11/2026.
3. Los modelos y la API de Pollinations cambian con frecuencia; los modelos sugeridos deben revisarse.
4. Sin sincronización entre dispositivos: el historial es solo local.
5. SensitiveDataDetector es heurístico (puede tener falsos positivos y negativos).
6. Las pantallas no tienen pruebas automáticas de interfaz.

7. ApplicationId sigue con el valor de plantilla com.companyname.asistenteia.

21.4 Trabajo futuro

Prioridad	Mejora
Alta	Migrar a **.NET 10 LTS** y actualizar paquetes (CommunityToolkit.Mvvm, xUnit).
Alta	Ejecutar y documentar con capturas los casos P10–P20 y las 66 pruebas automáticas.
Media	Respuestas en *streaming* para ver el texto mientras se genera.
Media	Conversaciones de varios turnos (enviar el historial de la conversación como contexto).
Media	Cambio automático de proveedor opcional ante 5xx/429, siempre informando al usuario.
Media	Exportar el historial a PDF o Markdown.
Media	Pruebas de interfaz y pruebas de AiService con un HttpMessageHandler simulado.
Baja	Publicar en Microsoft Store / Google Play con un ApplicationId propio.
Baja	Backend propio para no exponer claves si la app se distribuye a muchos usuarios.